

Arquiteturas de Armazenamento Físico para Sistemas de Banco de Dados

NAS (*Network Attached Storage*) e SAN (*Storage Area Network*)

Estudo comparativo, protocolos, *tiering* automatizado,
armazenamento por objetos e recomendações de uso

Integrantes do grupo

Nome completo	DRE
Bernardo Brandão Pozzato Carvalho Costa	123289593
Enzo de Carvalho Sampaio	123386206
Gabriel Schmitz Corrêa Rizawinsk	123225573
Guilherme En Shih Hu	123224674
Raphael Henrique da Silva Pereira	123311073
Vivian Maria da Silva e Souza	123205793

Resumo

Este trabalho compara as duas arquiteturas de armazenamento em rede que sustentam a maior parte das instalações corporativas de Sistemas de Banco de Dados (SBD): **NAS**, que exporta uma *semântica de arquivo* sobre a rede IP de propósito geral, e **SAN**, que exporta uma *semântica de bloco* sobre uma rede dedicada de armazenamento. A distinção não é de desempenho nem de meio físico — é de *unidade de abstração*, e é dela que decorrem todas as demais diferenças: quem é dono do sistema de arquivos, onde ocorre o travamento, quem garante a atomicidade da página e o que acontece quando a rede oscila.

Cobrimos os protocolos de cada família com suas normas de origem: SMB/CIFS, NFS e AFP no lado NAS; FCP sobre *Fibre Channel* (nas topologias ponto-a-ponto, *arbitrated loop* e *fabric* comutada), iSCSI, FCIP e FCoE no lado SAN. Discutimos *Automated Storage Tiering* (AST) e *Object-Based Storage*, e fechamos com uma recomendação estruturada de quando adotar cada solução, considerando explicitamente como o núcleo do SGBD (*Database Engine*) usa o armazenamento em **nível secundário** (*online*) e em **nível terciário** (*offline*).

Todo número publicado carrega proveniência explícita, classificada em quatro categorias (especificação normativa, *datasheet* de fabricante, medição publicada, derivação nossa), com uma quinta categoria legítima: *não encontrado*. O trabalho passou por uma rodada de verificação adversarial documentada na Seção 13, e registra **cinco divergências no material-fonte da disciplina** — um erro de unidade e uma imprecisão de mecanismo no Silberschatz *et al.*, um erro de definição, uma imprecisão e um dado desatualizado por um fator de dezesseis no Elmasri & Navathe — além de duas divergências em fontes de fabricante: a especificação do LTO-10, cuja taxa comprimida não fecha com a razão de compressão que ela mesma declara, e as três escalas distintas sob as quais o valor “128GFC” é publicado.

Palavras-chave: NAS; SAN; *Fibre Channel*; iSCSI; NFS; SMB; FCoE; FCIP; *Automated Storage Tiering*; armazenamento por objetos; armazenamento secundário; armazenamento terciário.

Conteúdo

Resumo	2
1 Introdução	5
1.1 Motivação	5
1.2 Objetivos	5
1.3 Metodologia e critérios de proveniência	5
1.4 Papel de cada fonte	6
2 Fundamentos: o que o SGBD exige do armazenamento	6
2.1 A hierarquia de armazenamento e os níveis secundário e terciário	6
2.2 O que o <i>Database Engine</i> realmente pede ao armazenamento	7
2.3 Por que a rede entrou no caminho do dado	7
3 NAS — <i>Network Attached Storage</i>	8
3.1 Como funciona	8
3.2 Consequências diretas da semântica de arquivo	9
3.3 Protocolo NAS I — SMB/CIFS	9
3.4 Protocolo NAS II — NFS	10
3.5 Protocolo NAS III — AFP (<i>Apple Filing Protocol</i>)	11
3.6 Vantagens e desvantagens do NAS	13
4 SAN — <i>Storage Area Network</i>	13
4.1 Como funciona	13
4.2 <i>Fibre Channel</i> : a pilha	14
4.3 Topologias FC — incluindo o “FC <i>Switch</i> ” do enunciado	16
4.4 Protocolo SAN I — FCP sobre <i>Fibre Channel</i>	17
4.5 Protocolo SAN II — iSCSI	18
4.6 Protocolo SAN III — FCIP	19
4.7 Protocolo SAN IV — FCoE	20
4.8 Dois mecanismos que a SAN pressupõe: <i>multipath</i> e RAID	20
4.9 Nota de fronteira: NVMe-oF	21
4.10 Vantagens e desvantagens da SAN	22
5 Quadro comparativo consolidado	22
5.1 NAS × SAN: as dimensões que realmente decidem	25
6 AST — <i>Automated Storage Tiering</i>	25
6.1 O que é e por que existe	25
6.2 Como funciona, concretamente	25
6.3 A tensão que ninguém menciona: AST versus <i>buffer pool</i>	26
6.4 Recomendação sobre AST	27
7 Object-Based Storage	27
7.1 O terceiro paradigma	27
7.2 Origem acadêmica	27
7.3 Propriedades: por que escala tanto	28

7.4	Por que ainda não serve como armazenamento primário de OLTP	29
7.5	Onde ele venceu, e venceu completamente	29
8	Recomendação geral	29
8.1	Princípio orientador	29
8.2	Nível secundário (<i>storage online</i>)	30
8.3	Nível terciário (<i>storage offline</i>)	31
9	Exemplos reais	33
10	Correções factuais ao material-fonte	34
11	Conclusão	34
12	Questões em aberto para o debate	35
	Referências	36
13	Post-Mortem	38
13.1	Como a IA foi usada, e como não foi	38
13.2	Log dos <i>prompts</i> -chave	38
13.3	Autoria: quem fez o quê, quem decidiu o quê e por quê	41
13.4	Erros gerados pela IA e correções aplicadas	42
13.5	Primeira rodada de verificação adversarial	43
13.6	Segunda rodada: simulação da correção do professor	44
13.7	O que a IA fez bem, para calibrar	45
A	Glossário de siglas	47

1 Introdução

1.1 Motivação

Silberschatz, Korth e Sudarshan abrem o Capítulo 12 lembrando que o modelo lógico é o nível correto para o *usuário* do banco pensar, mas que o projetista do SGBD não tem esse luxo: “o objetivo de um sistema de banco de dados é simplificar e facilitar o acesso aos dados”, e simplificar para o usuário significa *complicar* deliberadamente para quem implementa [2]. A escolha da arquitetura de armazenamento é o exemplo mais direto disso. Uma consulta SQL não muda de sintaxe quando o *tablespace* sai de um disco local e vai para um *array* conectado por *Fibre Channel*; mas o custo de cada acesso, a semântica de travamento, o comportamento sob falha e o próprio conjunto de configurações suportadas pelo fabricante do SGBD mudam completamente.

Elmasri e Navathe registram a causa econômica da migração para armazenamento em rede: “o custo total de gerenciar todos os dados está crescendo tão rapidamente que, em muitos casos, o custo de gerenciar o armazenamento conectado ao servidor excede o custo do próprio servidor”, e o custo de *aquisição* do armazenamento é “apenas uma pequena fração — tipicamente apenas 10 a 15% — do custo geral de gerenciamento de armazenamento” [1]. Ou seja: a discussão NAS $\times$ SAN nunca foi só sobre velocidade. Foi, desde o início, sobre desacoplar a capacidade do servidor que a consome.

1.2 Objetivos

O enunciado da tarefa fixa uma lista de tópicos obrigatórios. Adotamos essa lista como *checklist literal* e a reproduzimos na Tabela 1, indicando onde cada item é tratado. Um avaliador que confira item a item deve conseguir localizar cada exigência sem ler o texto corrido.

Tabela 1: Checklist literal do enunciado e sua localização neste relatório.

Item exigido pelo enunciado	Onde está
Como funciona cada uma das soluções de armazenamento	Seções 3 e 4
Protocolo NAS: SMB/CIFS	Seção 3.3
Protocolo NAS: NFS sobre redes TCP/IP	Seção 3.4
Protocolo NAS: AFP (<i>Apple Filing Protocol</i>)	Seção 3.5
Protocolo SAN: iSCSI (<i>Internet SCSI</i>)	Seção 4.5
Protocolo SAN: baseados em FC (<i>Fibre Channel</i>)	Seções 4.2 e 4.4
Variação: FC <i>Switch</i> (<i>fabric</i> comutada)	Seção 4.3
Variação: FCIP (<i>Fibre Channel over IP</i>)	Seção 4.6
Variação: FCoE (<i>Fibre Channel over Ethernet</i>)	Seção 4.7
Comentar a técnica AST (<i>Automated Storage Tiering</i>)	Seção 6
Comentar <i>Object-Based Storage</i>	Seção 7
Características, vantagens, desvantagens comparativas	Seções 3.6, 4.10 e 5
Recomendação geral: quando optar por um ou por outro	Seção 8
considerando o uso pelo núcleo do SDB (<i>Database Engine</i>)	Seções 2.2 e 8.2
em nível secundário (<i>storage online</i>)	Seção 8.2
em nível terciário (<i>storage offline</i>)	Seção 8.3
Exemplos reais	Seção 9
Post-Mortem: log dos <i>prompts</i> -chave	Seção 13.2
Post-Mortem: demonstração de autoria (quem fez/decidiu o quê)	Seção 13.3
Post-Mortem: erros gerados pela IA e correções aplicadas	Seção 13.4
Folha de rosto com nome completo e DRE	p. 1

1.3 Metodologia e critérios de proveniência

Adotamos uma regra única e rígida: **todo número publicado neste relatório declara sua origem**, em uma de cinco categorias.

- (a) **Especificação normativa** — valor fixado por uma norma (RFC do IETF, padrão T11/INCITS, documento aberto da Microsoft). É o grau mais forte: não muda com o produto.
- (b) **Datasheet de fabricante** — valor publicado pelo fabricante para um modelo identificado. Vale para *aquele* modelo, não para a categoria.

- (c) **Medição publicada por terceiro** — número obtido em artigo revisado por pares ou publicação técnica que descreve a metodologia.
- (d) **Derivação nossa** — conta feita por nós a partir de (a), (b) ou (c). A conta aparece explicitamente no texto.
- (e) **Não encontrado** — declaramos a lacuna. Não preenchemos com estimativa disfarçada de dado.

Além da origem, verificamos sistematicamente o *rótulo*, porque o erro mais difícil de detectar é um número certo descrevendo outra coisa. As armadilhas que efetivamente encontramos nesta tarefa foram: (i) taxa **full-duplex** versus taxa **por direção** — diferença de exatamente $2\times$ que a FCIA e a norma T11 publicam de formas diferentes (Seção 4.2); (ii) **bit versus byte**, que produziu um erro no próprio livro do Silberschatz (Seção 10); (iii) **cliente versus servidor** de um protocolo, que muda a data de descontinuação do AFP em cinco anos (Seção 3.5); e (iv) **taxa bruta de linha versus taxa útil**, que depende da codificação de linha (8b/10b ou 64b/66b).

Números voláteis (preços, prazos de recuperação, participação de mercado) recebem carimbo de data. Um preço sem data não é dado.

1.4 Papel de cada fonte

Seguindo a orientação de não citar por citar, atribuímos papéis distintos às três fontes bibliográficas da disciplina:

- **Elmasri & Navathe, Cap. 16** [1] — é a fonte *primária* desta tarefa. A Seção 16.11 (“*Modern Storage Architectures*”) é o único trecho dos três livros que trata explicitamente de SAN, NAS, iSCSI, FCIP, FCoE, AST e armazenamento por objetos. Dela extraímos a taxonomia e o vocabulário.
- **Silberschatz, Korth & Sudarshan, Cap. 12** [2] — fornece a *física do meio* e a definição canônica dos níveis secundário e terciário (Seção 2.1), que é o eixo pedido pelo enunciado na recomendação final. A Seção 12.2 trata SAN e NAS em dois parágrafos, mas define com precisão a interface orientada a bloco.
- **Garcia-Molina, Ullman & Widom, Cap. 13** [3] — fornece a *formalização do modelo de custo* de E/S: a ideia de que o custo de uma consulta se mede em número de acessos a bloco, e que a latência do acesso domina o cálculo. É esse arcabouço que usamos na Seção 2.2 para explicar por que a diferença entre NAS e SAN se manifesta em transações curtas e praticamente desaparece em varreduras longas.

2 Fundamentos: o que o SGBD exige do armazenamento

2.1 A hierarquia de armazenamento e os níveis secundário e terciário

O enunciado pede explicitamente a recomendação “tanto em nível secundário (*storage online*) como em nível terciário (*storage offline*)”. Essa terminologia é a de Silberschatz *et al.*, e vale reproduzi-la com exatidão, porque ela organiza toda a Seção 8:

Definição normativa dos níveis (Silberschatz *et al.*, §12.1)

“As mídias de armazenamento mais rápidas — por exemplo, *cache* e memória principal — são chamadas de **armazenamento primário**. As mídias no próximo nível da hierarquia — por exemplo, memória *flash* e discos magnéticos — são chamadas de **armazenamento secundário**, ou *online storage*. As mídias no nível mais baixo da hierarquia — por exemplo, fita magnética e *jukeboxes* de disco óptico — são chamadas de **armazenamento terciário**, ou *offline storage*.” [2]

Elmasri e Navathe usam a mesma divisão em três níveis e acrescentam o critério operacional que mais importa aqui: “dados em armazenamento secundário ou terciário não podem ser processados diretamente pela CPU; primeiro precisam ser copiados para o armazenamento primário” [1]. A consequência prática é que **NAS e SAN são, ambos, tecnologias de nível secundário** — os

dois entregam armazenamento *online*, endereçável, disponível sem intervenção do operador. O armazenamento por objetos, por sua vez, atravessa a fronteira: em suas classes de acesso imediato ele é secundário; em suas classes de arquivamento profundo, com prazos de recuperação medidos em horas, ele funciona como terciário sem fita.

2.2 O que o *Database Engine* realmente pede ao armazenamento

Antes de comparar arquiteturas, é preciso ser explícito sobre o que o núcleo de um SGBD exige. Cinco requisitos, em ordem de rigidez:

1. **Acesso em unidades de bloco/página.** Silberschatz define: “um *drive* de estado sólido (SSD) usa memória *flash* internamente para armazenar dados, mas provê uma interface similar à de um disco magnético, permitindo que dados sejam armazenados ou recuperados em unidades de um bloco; tal interface é chamada de **interface orientada a bloco**” [2]. Todo o modelo de custo de Garcia-Molina *et al.* [3] é construído sobre essa unidade. Tamanhos típicos de bloco de banco de dados: 8 KiB no PostgreSQL e no Oracle (*default*), 16 KiB no InnoDB do MySQL.
2. **Durabilidade sob comando explícito.** O *commit* de uma transação só pode retornar depois que o registro de *log* correspondente estiver em mídia não volátil. Isso é implementado por uma chamada de sincronização (*fsync*, *fdatsync*, ou escrita com *O_DSYNC*). A **latência dessa chamada**, e não a banda do canal, é o que limita a taxa de *commits* de uma carga OLTP.
3. **Atomicidade da escrita de página.** Se a energia cai no meio da gravação de uma página de 16 KiB, o SGBD precisa poder detectar e reparar a página parcialmente escrita (*torn page*). O manual do MySQL descreve o mecanismo: “o *doublewrite buffer* é uma área de armazenamento onde o InnoDB escreve as páginas descarregadas do *buffer pool* antes de escrevê-las em suas posições próprias nos arquivos de dados”, de modo que “se houver uma saída inesperada do sistema operacional, do subsistema de armazenamento ou do processo *mysqld* no meio da escrita de uma página, o InnoDB pode encontrar uma cópia boa da página no *doublewrite buffer* durante a recuperação” [19]. O PostgreSQL resolve o mesmo problema por outro caminho, com *full_page_writes* no WAL. Ambos os mecanismos custam banda de escrita — e ambos podem ser desligados quando o dispositivo garante escrita atômica, o que o manual do MySQL menciona explicitamente para dispositivos com suporte a *atomic writes* [19].
4. **Semântica de travamento previsível.** Em configurações de *cluster* com armazenamento compartilhado (Oracle RAC, por exemplo), vários nós escrevem no mesmo conjunto de blocos. O travamento tem de ser correto sob falha de nó e sob partição de rede.
5. **Comportamento determinado sob indisponibilidade transitória.** Se o caminho até o armazenamento some por 30 segundos, o SGBD precisa saber se a operação de E/S vai bloquear, retornar erro, ou retornar sucesso falso. A diferença entre bloquear e errar decide se a instância sobrevive ou aborta.

É contra esses cinco requisitos — e não contra um *benchmark* genérico — que NAS e SAN devem ser avaliados.

2.3 Por que a rede entrou no caminho do dado

O caminho histórico é DAS → NAS/SAN. No modelo **DAS** (*Direct Attached Storage*), os discos são cabeados ao servidor pelas interfaces SATA, SAS ou NVMe descritas por Silberschatz na Seção 12.2 [2]. É simples, é o de menor latência, e tem dois defeitos estruturais que Elmasri e Navathe apontam: a capacidade fica presa a um servidor específico — “muitos usuários de sistemas RAID não conseguem usar a capacidade efetivamente porque ela precisa estar ligada de maneira fixa a um ou mais servidores” — e o custo de gerenciamento explode com o número de ilhas [1].

A resposta da indústria foi mover a fronteira de rede para dentro do caminho de E/S, e há exatamente duas maneiras de fazer isso, que geram as duas arquiteturas deste trabalho.

A distinção fundamental, em uma frase

SAN move a fronteira de rede *abaixo* do sistema de arquivos: o servidor recebe blocos e continua sendo o dono do sistema de arquivos. **NAS** move a fronteira *acima* do sistema de arquivos: o servidor recebe arquivos, e o sistema de arquivos é do dispositivo de armazenamento. Tudo o mais — protocolo, cabo, latência, custo, modelo de falha — é consequência dessa escolha.

A Figura 1 mostra onde essa fronteira cai em cada arquitetura.

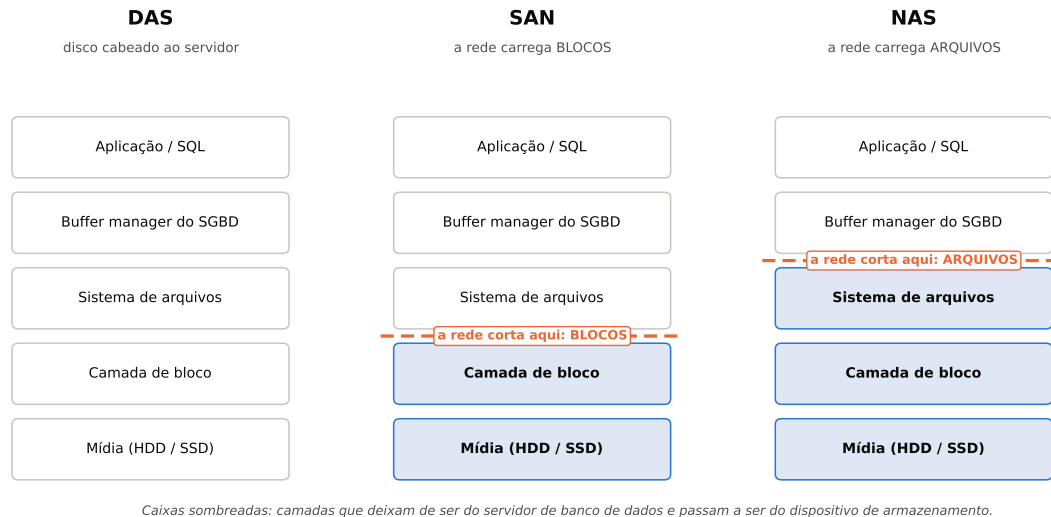


Figura 1: Onde a rede corta a pilha de E/S. Em DAS não há corte; em SAN o corte é abaixo do sistema de arquivos (tráfego de blocos SCSI ou NVMe); em NAS o corte é acima (tráfego de operações de arquivo). O componente sombreado é o que passa a ser responsabilidade do dispositivo de armazenamento, e não mais do servidor de banco de dados.

3 NAS — *Network Attached Storage*

3.1 Como funciona

Elmasri e Navathe descrevem o NAS de maneira quase provocativa: os dispositivos NAS “são, de fato, servidores que não fornecem nenhum dos serviços comuns de servidor, mas simplesmente permitem a adição de armazenamento para compartilhamento de arquivos”. Um único dispositivo de *hardware*, “frequentemente chamado de *NAS box* ou *NAS head*, atua como a interface entre o sistema NAS e os clientes de rede”, e “os clientes se conectam ao *NAS head* em vez de aos dispositivos de armazenamento individuais” [1].

Operacionalmente, a sequência é a seguinte:

1. O *NAS head* possui discos (ou SSDs) organizados internamente, tipicamente em RAID. Elmasri e Navathe registram que “tais dispositivos tipicamente suportam RAID níveis 0, 1 e 5” [1].
2. Sobre esse armazenamento, o *NAS head* **monta e opera um sistema de arquivos próprio** (WAFL na NetApp, ZFS em muitos produtos, ext4/XFS em soluções abertas).
3. Esse sistema de arquivos é exportado pela LAN por meio de um protocolo de compartilhamento de arquivos — SMB/CIFS, NFS ou AFP.
4. O cliente (o servidor de banco de dados) monta o compartilhamento e enxerga *arquivos e diretórios*. Ele emite operações de arquivo (OPEN, READ, WRITE, CLOSE, LOCK), não comandos de bloco.

O ponto que decide tudo está no item 2: **quem executa a alocação de blocos, o *journaling* de metadados e o controle de concorrência de arquivo é o dispositivo NAS, não o servidor**. O servidor delega. Silberschatz resume: “NAS é muito parecido com SAN, exceto que, em vez de o armazenamento em rede aparecer como um disco grande, ele provê uma interface de sistema de arquivos usando protocolos de sistema de arquivos em rede como NFS ou CIFS” [2].

3.2 Consequências diretas da semântica de arquivo

- **Compartilhamento entre clientes é nativo.** Como o dispositivo é o dono do sistema de arquivos, ele arbitra o acesso concorrente. Vários servidores podem montar o mesmo compartilhamento sem *software* adicional. Em SAN, dois servidores que montam o mesmo LUN com um sistema de arquivos comum destroem os dados, a menos que se use um sistema de arquivos de *cluster* (GFS2, OCFS2, VMFS) ou um gerenciador de volume ciente de *cluster*.
- **Independência de sistema operacional do cliente.** Elmasri e Navathe: “sistemas NAS alegam maior independência de sistema operacional dos clientes” [1].
- **Coerência de *cache* é o problema difícil.** O NFS não oferece coerência forte por padrão. O manual do Linux é explícito: “o protocolo NFS não foi projetado para suportar coerência de *cache* verdadeira de sistema de arquivos de *cluster* sem algum tipo de serialização pela aplicação. Se coerência absoluta de *cache* entre clientes for necessária, as aplicações devem usar travamento de arquivo”, e “alternativamente, as aplicações podem abrir seus arquivos com a *flag* `O_DIRECT` para desabilitar completamente o *cache* de dados” [12]. Esta é exatamente a razão pela qual SGBDs suportados sobre NFS usam `O_DIRECT` ou um cliente NFS próprio (ver Seção 3.4).

3.3 Protocolo NAS I — SMB/CIFS

O que é. SMB (*Server Message Block*) é o protocolo de compartilhamento de arquivos nativo do mundo Windows. CIFS (*Common Internet File System*) **não é um protocolo diferente**: é o nome que a Microsoft deu, em 1996, à família de dialetos hoje chamada SMB 1. A documentação da própria Microsoft é explícita ao afirmar que o protocolo CIFS é um *dialeto* do SMB. Tratar “SMB” e “CIFS” como dois protocolos distintos é um erro comum; a leitura correta é que **CIFS é SMB 1**, e que tudo posterior a isso se chama SMB 2 ou SMB 3. O enunciado da tarefa usa a grafia composta “SMB/CIFS”, que é a convenção da indústria e a adotada aqui.

Modelo. É um protocolo de requisição-resposta orientado a sessão, historicamente sobre NetBIOS (porta 139) e, desde o Windows 2000, diretamente sobre TCP na **porta 445**. As operações são de alto nível: abrir arquivo, ler intervalo de *bytes*, travar intervalo de *bytes*, enumerar diretório, consultar atributos.

Evolução dos dialetos. A Microsoft documenta as versões na especificação aberta [MS-SMB2]. Os marcos relevantes:

Tabela 2: Evolução dos dialetos SMB. Categoria de proveniência: documentação da Microsoft.

Dialeto	Introduzido em	O que trouxe de relevante
SMB 1.0 (CIFS)	Anos 1980–1990	Protocolo original; “verboso”, muitas idas e voltas por operação.
SMB 2.0	Windows Vista / Server 2008	Reescrita: conjunto de comandos reduzido, requisições compostas, créditos de fluxo, identificadores de 64 bits.
SMB 2.1	Windows 7 / Server 2008 R2	<i>Leasing</i> de arquivo (<i>cache</i> de cliente mais agressivo e correto).
SMB 3.0	Windows 8 / Server 2012	SMB Direct (RDMA), SMB Multichannel , <i>transparent failover</i> , compartilhamentos continuamente disponíveis, criptografia fim-a-fim (AES-128-CCM). É a versão que torna SMB viável para banco de dados.
SMB 3.0.2	Windows 8.1 / Server 2012 R2	Otimizações de E/S pequena e aleatória; possibilidade de remover SMB1 completamente.
SMB 3.1.1	Windows 10 / Server 2016	Negociação pré-autenticação com integridade protegida; criptografia AES-128-GCM (AES-256-GCM só a partir do Windows 11 / Server 2022).

Estado do SMB 1. A Microsoft documenta que “a partir do *Windows 10 Fall Creators Update* e do Windows Server 2019, o SMBv1 não é mais instalado por padrão”, e que “o SMBv1 não é instalado por padrão em nenhuma edição do Windows 11 ou do Windows Server 2019 e versões posteriores” [11]. **Precisão de rótulo:** o primeiro *Windows Server* sem SMBv1 por padrão foi a versão 1709 do canal semianual; o Server 2019 é o primeiro do canal de suporte prolongado (LTSC).

Consequência prática para NAS: equipamentos antigos que só falam CIFS/SMB1 deixam de ser montáveis por clientes modernos sem reabilitação manual de um protocolo com vulnerabilidades conhecidas. Ao citar “SMB/CIFS” num projeto novo em 2026, o que se está de fato especificando é SMB 3.

Uso com SGBD. A Microsoft suporta oficialmente colocar arquivos de banco de dados do SQL Server em compartilhamento SMB: “o SQL Server 2012 (11.x) e versões posteriores” suportam armazenar bancos de dados de sistema (`master`, `model`, `msdb`, `tempdb`) e de usuário em *fileshare* SMB, tanto em instalação isolada quanto em *failover cluster* [18]. As condições que a documentação impõe são instrutivas porque expõem exatamente os pontos frágeis do NAS para banco de dados:

- para cargas críticas, “o compartilhamento de arquivos deve suportar *SMB 3.0 transparent failover* (disponibilidade contínua)” [18] — isto é: o protocolo precisa sobreviver à troca de nó do servidor de arquivos sem devolver erro ao SGBD;
- a conta de serviço precisa de `FULL CONTROL` no compartilhamento e no NTFS;
- **FILESTREAM não é suportado** em compartilhamento SMB;
- caminhos de *loopback*, compartilhamentos administrativos (`\\servidor\x$`) e unidades mapeadas não são suportados — só caminhos UNC.

3.4 Protocolo NAS II — NFS

O que é. O NFS (*Network File System*) é o padrão aberto de compartilhamento de arquivos do mundo UNIX/Linux, originalmente da Sun Microsystems (1984) e hoje mantido pelo IETF. É o protocolo NAS que efetivamente aparece em instalações sérias de banco de dados.

Versões e o que muda em cada uma. A Tabela 3 traz a linha do tempo com as normas. Proveniência: RFCs do IETF (especificação normativa).

Tabela 3: Versões do NFS. Fonte: RFCs do IETF. **Atenção à leitura:** a coluna “norma vigente” traz a RFC *atualmente em vigor*, que muitas vezes é uma reedição posterior — por isso a v4.2 (2016) aparece com data anterior à da v4.1 (2020). A cronologia real do protocolo está na coluna “introduzida em”.

Versão	Introduzida em	Norma vigente	Característica determinante
NFSv2	1989	RFC 1094	Sobre UDP; <i>offsets</i> de 32 bits (limite de 2 GiB por arquivo).
NFSv3	1995	RFC 1813	<i>Offsets</i> de 64 bits, escritas assíncronas com COMMIT explícito, TCP. Sem estado: o travamento fica <i>fora</i> do protocolo, num serviço separado (NLM, <i>Network Lock Manager</i>), o que torna a recuperação após falha frágil.
NFSv4.0	2000 (RFC 3010)	RFC 7530 (2015)	Com estado. Travamento, montagem e ACLs integrados ao protocolo; operações compostas (COMPOUND) reduzem idas e voltas; uma única porta (2049), o que o torna atravessável por <i>firewall</i> ; segurança via RPCSEC_GSS.
NFSv4.1	2010 (RFC 5661)	RFC 8881 (2020)	Modelo de sessões , que dá semântica de execução exatamente uma vez (<i>exactly-once semantics</i>): a garantia vem do par sessão + <i>reply cache</i> e vale para requisições COMPOUND precedidas de uma operação SEQUENCE (ou CB_SEQUENCE) — não é incondicional. E pNFS (<i>parallel NFS</i>), que separa o caminho de metadados do caminho de dados, permitindo que o cliente leia dados diretamente de vários servidores em paralelo [4].
NFSv4.2	2016	RFC 7862	Cópia no lado do servidor, arquivos esparsos, <i>hole punching</i> .

Correção de rótulo: RFC 8881 obsoleta a RFC 5661

Boa parte da literatura ainda cita a RFC 5661 (2010) como a especificação do NFSv4.1. Ela foi **obsoletada pela RFC 8881, de 2020** [4]. Citar a RFC 5661 hoje aponta para um documento superado — ainda que o conteúdo técnico seja em grande parte o mesmo, a referência correta é a RFC 8881.

O calcanhar de Aquiles: coerência de *cache*. O NFS implementa *close-to-open consistency*: o cliente verifica a validade do *cache* ao abrir o arquivo e descarrega as alterações pendentes ao fechá-lo [12]. Isso é adequado para arquivos de escritório e desastroso para um arquivo de dados de banco aberto por semanas sem nunca ser fechado. É por isso que todo SGBD suportado sobre NFS ou usa `O_DIRECT`, ou substitui o cliente NFS do sistema operacional pelo seu próprio.

Oracle Direct NFS (dNFS). A Oracle escolheu o caminho radical: “o cliente Direct NFS (dNFS) integra a funcionalidade de cliente NFS diretamente no *software* Oracle para otimizar o caminho de E/S entre a Oracle e o servidor NFS” [17]. Ou seja, o SGBD implementa NFS *dentro de si mesmo*, falando diretamente com o servidor NFS por *sockets* próprios e ignorando o cliente NFS do *kernel*. Isso lhe dá controle sobre profundidade de fila, número de conexões TCP e semântica de *cache*. A Oracle documenta que “o cliente Direct NFS suporta os protocolos NFSv3, NFSv4, NFSv4.1 e pNFS”, e que “se o Oracle Database não conseguir se conectar a um servidor NFS usando o cliente Direct NFS, então ele se conecta usando o cliente NFS do *kernel* do sistema operacional” [17].

Por que o dNFS importa para o argumento deste trabalho

A existência do dNFS é a melhor evidência empírica da tese central: NAS entrega uma abstração *alta demais* para um SGBD. O maior fornecedor de bancos de dados relacionais do mercado achou necessário reimplementar o protocolo NAS dentro do próprio motor para poder usá-lo. Isso não significa que NAS não sirva para banco — significa que serve *quando o motor participa da decisão*, não quando ele é tratado como uma aplicação qualquer.

3.5 Protocolo NAS III — AFP (*Apple Filing Protocol*)

O que é. O AFP é o protocolo proprietário de compartilhamento de arquivos da Apple, originado no *AppleTalk Filing Protocol* de 1985–1988. Sua característica distintiva era preservar corretamente as particularidades do sistema de arquivos do Macintosh — *resource forks*, tipo e criador de arquivo, nomes longos com acentuação, e metadados que o SMB1 da época mutilava. Por muitos anos foi o único jeito de fazer um NAS servir Macs sem corromper arquivos, e era o protocolo obrigatório para *backups* do Time Machine em rede.

Como funciona. O AFP é um protocolo de sessão, com estado, historicamente transportado sobre AppleTalk (ASP/ATP) e, desde o AFP 3.0, sobre TCP na **porta 548** por meio do **DSI** (*Data Stream Interface*), a camada que enquadra requisições e respostas e multiplexa comandos numa conexão TCP. Suas características determinantes, e o que elas explicam:

- **Modelo de sessão com estado.** O cliente faz `FPLOGIN`, abre um *volume* com `FPOPENVOL`, e daí em diante opera sobre identificadores de sessão. Falha de rede derruba a sessão; o AFP 3.x acrescentou reconexão para mitigar isso.
- **Bifurcação de arquivo (*forks*).** O AFP foi desenhado em torno do sistema de arquivos do Macintosh, em que um arquivo tem um *data fork* e um *resource fork* — e os comandos são explicitamente bifurcados: `FPOPENFORK`, `FPREAD/FPWRITE` sobre um *fork* identificado. Era esta a razão técnica de existir: SMB1 e NFS não tinham como representar o *resource fork* sem truques (arquivos `._nome`, *AppleDouble*), e o resultado era corrupção silenciosa de metadados.
- **Metadados nativos do Mac** — tipo e criador de arquivo, *Finder info*, posição de ícone — transportados como atributos de primeira classe, não como emulação.
- **Travamento por intervalo de *bytes*** via `FPBYTERANGELOCK`, mas com semântica de sessão: o travamento morre com a sessão, e não há o equivalente às *leases* com prazo do NFSv4 ou aos *oplocks/leases* do SMB 2/3.
- **Nomes e codificação:** suporte nativo a nomes longos em UTF-8 com decomposição canônica (a forma NFD que o macOS usa), outro ponto em que os protocolos concorrentes da época

falhavam.

O que ele nunca teve — e que é a razão de não servir a banco de dados: nenhum caminho de dados com RDMA, nenhum equivalente a *multichannel*, nenhuma noção de disponibilidade contínua com *failover* transparente, e um travamento amarrado à sessão. Comparado ao SMB 3 (Seção 3.3) e ao NFSv4.1 (Seção 3.4), o AFP parou no conjunto de recursos de um protocolo de compartilhamento de arquivos de escritório.

Estado atual — e é aqui que a maior parte do material disponível erra. O AFP está em processo de remoção do macOS, em duas etapas separadas por cerca de cinco anos:

- o **servidor** AFP (a capacidade de um Mac *compartilhar* pastas por AFP) foi removido no macOS 11 Big Sur (2020);
- o **cliente** AFP (a capacidade de um Mac *montar* um compartilhamento AFP de um NAS) foi **depreciado no macOS 15.5 Sequoia**, em maio de 2025. O texto da Apple é direto: “o cliente do *Apple Filing Protocol* (AFP) está depreciado e será removido em uma versão futura do macOS” [20];
- essa versão futura chegou: o **macOS 27 “Golden Gate”**, cujos *betas* de desenvolvedor, a partir de junho de 2026, **já não incluem o cliente AFP**, com o efeito colateral de encerrar o suporte a *backups* do Time Machine em Time Capsules e AirPort Disks [28].

Armadilha de rótulo: cliente $\neq$ servidor — e correção da nossa própria primeira versão

Dizer “a Apple removeu o AFP no Big Sur” é meia-verdade que erra por cinco anos: removeu o *servidor*; o *cliente* sobreviveu até 2026.

Registro de erro nosso. A primeira versão deste relatório afirmava que “a versão de remoção efetiva ainda não foi anunciada”. Isso era verdade em maio de 2025 e **já não era verdade em setembro de 2026**: a rodada de verificação adversarial (Seção 13.5) localizou a cobertura de junho de 2026 documentando a ausência do cliente AFP nos *betas* do macOS 27 [28], além do próprio aviso exibido pelo macOS 26 Tahoe sobre o fim do suporte a Time Capsule na versão seguinte.

Ressalva de proveniência, mantida: não localizamos comunicado formal de imprensa da Apple declarando a remoção. A evidência é (i) o texto de depreciação da própria Apple [20], (ii) o aviso do sistema operacional no macOS 26 e (iii) a verificação dos *betas* do macOS 27 reportada pela imprensa especializada [28]. **Categoria:** (ii) é fonte primária; (iii) é medição publicada por terceiro.

Relevância para SBD: nenhuma, e isso é o achado. Alcance da afirmação: pesquisamos as matrizes de suporte publicadas por Oracle, Microsoft, PostgreSQL e MySQL e não localizamos AFP em nenhuma delas; o que afirmamos, portanto, é que *nenhum dos quatro SGBDs de maior participação o lista como configuração suportada*, e não um negativo universal sobre todo o mercado. O motivo técnico está na Seção anterior: sem travamento independente de sessão, sem RDMA e sem disponibilidade contínua, o AFP não atende aos requisitos 4 e 5 da Seção 2.2. O protocolo nunca teve travamento em intervalo de *bytes* adequado a cargas transacionais, nunca teve RDMA nem múltiplos canais, e agora chegou ao fim da vida. A recomendação técnica correta, em 2026, é: **para um NAS que precise servir clientes Apple, use SMB 3**, que é o protocolo primário do macOS desde o OS X 10.9 Mavericks (2013) — a transição já tinha treze anos quando o AFP foi finalmente retirado. Se o AFP aparece na lista do enunciado, é como item de completude histórica; é mais honesto dizer isso do que fabricar uma aplicação de banco de dados para ele.

3.6 Vantagens e desvantagens do NAS

Declaração de lacuna: os itens de custo desta tabela não têm número

As linhas de custo da Tabela 4 e das tabelas comparativas seguintes — “menor custo por TB útil”, “custo relativo menor/maior” — são **afirmações estruturais, não medições**. Elas se sustentam em um fato verificável (o NAS dispensa HBA, *switch* FC, transceptores e cabeamento dedicados, que a SAN exige) e em outro que *não* conseguimos sustentar com fonte: *quanto* isso representa em US\$/TB útil.

Aplicando a regra da Seção 1.3, a categoria correta é **não encontrado**. Não localizamos, para nenhum fornecedor, tabela pública de preço de porta FC ou de *array* unificado que permitisse uma comparação honesta e datada — preço de canal depende de volume e de contrato, e reproduzi-lo sem data seria violar a regra de que “número volátil sem data não é dado”.

O que se pode afirmar com rigor, e é o que afirmamos: a SAN acrescenta *categorias* de componente que o NAS não tem, e cada categoria acrescenta custo de aquisição, de manutenção e de competência da equipe. A *direção* da comparação é estrutural e certa; a *magnitude* não está neste trabalho. O Apêndice de aprofundamentos registra o levantamento de TCO como o item que fecharia essa lacuna.

Tabela 4: NAS: vantagens e desvantagens sob a ótica de um SBD. As linhas de custo são qualitativas — ver a declaração de lacuna acima.

Vantagens	Desvantagens
Usa a rede Ethernet/IP existente: sem HBA dedicada, sem <i>switch</i> FC, sem equipe especializada.	Compete por banda com o tráfego de aplicação, a menos que se segregue por VLAN ou rede física separada.
Compartilhamento concorrente entre servidores é nativo, sem sistema de arquivos de <i>cluster</i> .	Coerência de <i>cache</i> fraca por padrão (<i>close-to-open</i>); exige <code>O_DIRECT</code> , travamento explícito ou cliente dedicado [12].
Provisionamento simples: criar um compartilhamento e conceder permissão, sem zoneamento nem mapeamento de LUN.	Camada extra de sistema de arquivos no caminho: metadados, travamento e <i>journaling</i> do lado do NAS somam latência.
Independência de sistema operacional dos clientes [1].	Semântica de escrita atômica de página não é garantida pelo protocolo; o SGBD tem que se proteger sozinho.
Instantâneos, <i>clones</i> e replicação no nível de arquivo, com granularidade compreensível.	Nem todo recurso do SGBD funciona (FILESTREAM não é suportado sobre SMB [18]).
Custo por terabyte útil menor, tanto de aquisição quanto de operação.	Depende de recursos avançados do protocolo (SMB 3 <i>transparent failover</i> , NFSv4.1 <i>sessions</i>) para sobreviver a falhas sem derrubar a instância.

4 SAN — *Storage Area Network*

4.1 Como funciona

Silberschatz define com precisão: “na arquitetura de rede de área de armazenamento (SAN), um grande número de discos é conectado por uma rede de alta velocidade a um conjunto de computadores servidores. Os discos são geralmente organizados localmente usando uma técnica de organização de armazenamento chamada RAID, para dar aos servidores uma *visão lógica de um disco muito grande e muito confiável*” [2]. Elmasri e Navathe complementam com a motivação operacional: “em uma SAN, os periféricos de armazenamento *online* são configurados como nós em uma rede de alta velocidade e podem ser conectados e desconectados de servidores de maneira muito flexível” [1].

A sequência operacional, em contraste direto com a do NAS:

1. O *array* de armazenamento agrega discos físicos em *pools* protegidos por RAID ou por codificação de apagamento.
2. Do *pool*, o administrador recorta volumes lógicos e os apresenta como **LUNs** (*Logical Unit Numbers*) — o identificador SCSI de uma unidade lógica.
3. Uma rede dedicada (FC ou Ethernet segregada) transporta **comandos SCSI** (ou, mais recentemente, comandos NVMe) entre o iniciador no servidor e o alvo no *array*.

4. O sistema operacional do servidor vê um **dispositivo de bloco cru**, indistinguível de um disco local: `/dev/sdb`, `PhysicalDrive2`. Ele formata esse dispositivo com seu próprio sistema de arquivos, ou o entrega cru ao SGBD (é o que o Oracle ASM faz).

A frase de Elmasri e Navathe que melhor captura o efeito é: “aplicações de gerenciamento de armazenamento existentes podem ser portadas para configurações SAN usando redes *Fibre Channel* que encapsulam o protocolo SCSI legado. Como resultado, **os dispositivos ligados à SAN aparecem como dispositivos SCSI**” [1]. É uma ilusão deliberada, e é ela que faz a SAN funcionar: o servidor e o SGBD não precisam saber que existe uma rede no caminho.

4.2 *Fibre Channel*: a pilha

O *Fibre Channel* é padronizado pelo comitê **T11** do INCITS. Uma observação de nomenclatura que vale registrar: a grafia normativa é **Fibre** Channel, com *-re*, justamente para deixar claro que o padrão não exige fibra óptica — as primeiras variantes rodavam sobre cabo de cobre. O enunciado da tarefa escreve “FC (Fibre/Fiber Channel)”, reconhecendo as duas grafias; o livro do Elmasri alterna entre “Fibre Channel” e “Fiber Channel” no mesmo capítulo [1], e o do Silberschatz usa “Fiber Channel FC” [2]. A grafia do T11 é *Fibre*.

A pilha FC é dividida em cinco camadas:

Tabela 5: Camadas do *Fibre Channel*. Proveniência: arquitetura T11/INCITS.

Camada	Nome	Função
FC-0	Físico	Meio, conectores, transceptores, taxa de sinalização (<i>baud</i>).
FC-1	Codificação	Codificação de linha e sincronismo: 8b/10b até 8GFC; 64b/66b no 16GFC; 256b/257b com RS-FEC obrigatório a partir de 32GFC (RS(528,514) no 32GFC); sinalização PAM-4 a partir de 64GFC.
FC-2	Sinalização	Camada central : quadros, sequências, trocas (<i>exchanges</i>), classes de serviço e o controle de fluxo por créditos de buffer (BB_Credit).
FC-3	Serviços comuns	Serviços que atravessam múltiplas portas (<i>hunt groups</i> , <i>striping</i>). Pouco usada.
FC-4	Mapeamento	Mapeia protocolos superiores sobre FC. O relevante aqui é o FCP (<i>Fibre Channel Protocol</i>), o mapeamento de SCSI-3 sobre FC; e, mais recentemente, FC-NVMe .

Por que o controle de fluxo por créditos importa. Diferente do Ethernet clássico, que descarta quadros sob congestionamento e delega a recuperação ao TCP, o FC-2 usa **BB_Credit**: uma porta só transmite um quadro se tiver crédito, e o crédito é devolvido pela porta receptora quando ela libera um *buffer*. O resultado é uma rede *sem perdas por congestionamento*, e é isso que permite ao FC dispensar TCP e ainda assim entregar comandos SCSI de forma confiável. Como referência de magnitude, o *product brief* do *switch* Brocade G710 declara “2000 *buffers* de quadro alocados dinamicamente” [15]. **Proveniência:** *datasheet* de fabricante, modelo específico.

Endereçamento e zoneamento. Cada porta FC tem um **WWN** (*World Wide Name*), identificador global de 64 bits gravado de fábrica, análogo ao endereço MAC. Ao entrar na *fabric*, a porta faz FLOGI e recebe um **endereço de fabric de 24 bits** (Domain / Area / Port), usado para roteamento. O **zoneamento** (*zoning*), configurado no *switch*, define quais iniciadores podem ver quais alvos; é o controle de acesso da SAN, e o análogo funcional do *firewall* na rede IP. Complementarmente, o **LUN masking**, configurado no *array*, define quais LUNs cada iniciador enxerga. As duas camadas juntas são o que impede que o servidor de teste monte, por engano, o LUN de produção.

Velocidades: onde mora a armadilha de rótulo

Há duas maneiras de expressar a velocidade de um enlace FC, e confundi-las é a origem da maior parte dos erros que circulam sobre desempenho de *Fibre Channel*. A Tabela 6 reproduz o *roadmap*

vigente da FCIA, que desde a revisão de 2023 publica **duas tabelas separadas**: o FC serial (uma via) e o *Inter-Switch Link* (vias paralelas) [9].

Tabela 6: *Speedmap* vigente do *Fibre Channel* [9]. As colunas “FCIA”, “taxa de linha”, “T11” e “mercado” são reproduzidas da fonte; a coluna “por direção” é **derivação nossa** (metade do valor da FCIA), justificada logo abaixo. “Demanda” = “*market demand*” no original.

Produto	FCIA (MB/s)	Por direção (MB/s, derivado)	Taxa de linha (GBd)	Norma T11 concluída	Disponib. no mercado
<i>Fibre Channel serial</i>					
8GFC	1.600	800	8,5 NRZ	2006	2008
16GFC	3.200	1.600	14,025 NRZ	2009	2011
32GFC	6.400	3.200	28,05 NRZ	2013	2016
64GFC	12.800	6.400	28,9 PAM-4	2017	2020
128GFC	24.850	12.425	56,1 PAM-4	2022	2024
256GFC	49.700	24.850	112,2 PAM-4	2025	demanda
512GFC	TBD	TBD	TBD	2029	demanda
1TFC	TBD	TBD	TBD	2033	demanda
<i>Inter-Switch Link (vias paralelas)</i>					
10GFC	2.400	1.200	10,52 NRZ	2003	2009
128GFC	25.600	12.800	4 × 28,05 NRZ	2014	2016
256GFC	51.200	25.600	4 × 28,9 PAM-4	2018	2020

Prova de que os números da FCIA são *full-duplex* — e ela é de física, não de opinião

A FCIA **não declara** se seus valores são por direção ou somados. A única nota de rodapé do *roadmap* diz: “estes números são representativos de valores de vazão para a taxa de linha e dependem da carga útil” [9]. **Portanto, ler os valores como *full-duplex* é derivação nossa, não citação** — e é assim que apresentamos. A prova, porém, é dedutiva e não admite discussão: *uma direção não pode transportar mais bits do que a linha sinaliza*.

16GFC. A FCIA publica $3.200 \text{ MB/s} = 3.200 \times 8 = \mathbf{25,6 \text{ Gb/s}}$. A taxa de linha é 14,025 GBd em NRZ, isto é, **14,025 Gb/s brutos**. Como $25,6 > 14,025$, o valor publicado *não cabe* numa única direção.

128GFC. A FCIA publica $24.850 \text{ MB/s} = \mathbf{198,8 \text{ Gb/s}}$. A linha é 56,1 GBd em PAM-4 (2 bits por símbolo) = **112,2 Gb/s brutos**. Como $198,8 > 112,2$, idem.

Em ambos os casos o valor publicado excede a capacidade bruta da linha por um fator próximo de dois. **A única leitura fisicamente possível é a soma dos dois sentidos. Consequência prática:** ao dimensionar o caminho de E/S de um SGBD, use a coluna “por direção” — a leitura de um *tablespace* e a escrita do WAL não se somam num único sentido. Usar o número da FCIA sem dividir por dois superdimensiona o enlace em 100%.

O achado: a convenção de nomenclatura *quebra* na Gen 8, e há três números chamados “128GFC”

Até o 64GFC vale uma regra simples e muito repetida: X GFC entrega $X \times 100$ MB/s por direção — 16GFC $\rightarrow$ 1.600; 32GFC $\rightarrow$ 3.200; 64GFC $\rightarrow$ 6.400. Nessas gerações a razão entre o valor da FCIA e o da convenção é **exatamente 2,00**.

Na geração 8 a regra **falha**, e o motivo está documentado pela própria FCIA: “dobrar a velocidade teria exigido uma taxa de linha de 115,6 Gb/s, e as equipes técnicas não julgaram viável fechar o *link budget*” [14]. A linha ficou em **112,2 Gb/s**, e não nos 115,6 que o nome “128” sugere. Logo:

$$24.850 \div 2 = \mathbf{12.425} \text{ MB/s por direção} \neq 12.800 \text{ que o nome implica; } \frac{24.850}{12.800} = 1,941 \neq 2.$$

Derivação nossa que confirma a leitura. A eficiência útil sobre a linha bruta é a mesma nas duas gerações PAM-4: no 64GFC, 6.400 MB/s = 51,2 Gb/s sobre 57,8 brutos = 88,6%; no 128GFC, 12.425 MB/s = 99,4 Gb/s sobre 112,2 brutos = 88,6%. E a razão entre as vazões, $24.850/12.800 = 1,941$, é *idêntica* à razão entre as taxas de linha, $56,1/28,9 = 1,941$. **A FCIA escalou o 128GFC a partir do 64GFC pela taxa de linha**; o nome é que arredondou para cima.

Resultado: três números diferentes atendem por “128GFC”.

- **25.600 MB/s** — o 128GFC de *Inter-Switch Link*, quatro vias de 28,05 GBd NRZ. Esse, sim, soma 128 Gb/s de vias, e é de 2016.
- **24.850 MB/s** — o 128GFC serial de Gen 8, uma via de 56,1 GBd PAM-4, de 2024. É o que se compra hoje como “128GFC”.
- **12.800 MB/s** — o valor que a norma FC-PI-8 rev. 1.4 ainda publica como *Data Rate*. A FCIA registra, no seu Q&A, a pergunta “isso mudou?” e a resposta: “decidimos não mudar isso no documento FC” [14]. É um valor **herdado da convenção de nomenclatura**, deliberadamente não atualizado.

Quem cita “128GFC” sem dizer qual dos três está citando erra por até 2 $\times$.

Registro de erro nosso, e do que ele custou

A primeira versão deste relatório publicava o *speedmap* v21, de dezembro de 2016 [13], e declarava não ter conseguido acessar a tabela numérica vigente. **As duas coisas estavam erradas:** a tabela v24 (jul./2023) está disponível em HTML na página de *roadmap* da FCIA [9], e com ela vieram três correções — a separação entre FC serial e ISL, a disponibilidade do 64GFC em 2020 (e não 2019), e sobretudo o 128GFC serial de 24.850 MB/s, que *refuta* a afirmação de “fator exatamente 2” que aquela versão trazia como conclusão.

Por que registramos isso em vez de apagar. Declarar uma lacuna é resultado legítimo (Seção 1.3) — *declarar uma lacuna que não existe é o oposto disso*, e contamina todas as outras declarações de lacuna do trabalho. O erro foi encontrado na segunda rodada de verificação adversarial (Seção 13.6) e é o mais grave que cometemos.

Conferindo o Silberschatz contra a tabela

Silberschatz *et al.* escrevem que o *Fibre Channel* “suporta taxas de transferência de 1,6 a 12 *gigabytes* por segundo, dependendo da versão” [2]. Confrontando com a Tabela 6: 1,6 GB/s = 1.600 MB/s = 16GFC *por direção*; e 12 GB/s $\approx$ 12.800 MB/s = o valor *full-duplex* do 64GFC — ou, equivalentemente, o número herdado que o FC-PI-8 publica para o 128GFC. **A faixa do livro está correta para as gerações disponíveis quando a edição foi escrita**; o que ela não diz é qual das duas escalas está usando, e um leitor que a compare com o *speedmap* vai achar que o livro errou por um fator de 2. Registramos como *ambiguidade de rótulo*, não como erro. O erro de fato, no mesmo parágrafo, é outro, e está na Seção 10.

4.3 Topologias FC — incluindo o “FC Switch” do enunciado

O enunciado lista “FC Switch” como uma das variações a discutir. Tecnicamente, *FC Switch* não é um protocolo, e sim uma das três **topologias** definidas pelo T11. Elmasri e Navathe as descrevem como as “alternativas arquiteturais atuais para SAN” [1]:

Tabela 7: Topologias *Fibre Channel*.

Topologia	Descrição e situação atual
Ponto-a-ponto (FC-P2P)	Dois dispositivos ligados diretamente, sem <i>switch</i> . Banda integralmente dedicada, mas conectividade de dois nós apenas. Elmasri e Navathe a listam como a alternativa mais simples: “conexões ponto-a-ponto entre servidores e sistemas de armazenamento via <i>Fibre Channel</i> ” [1]. Uso residual: ligação direta de um servidor a um <i>array</i> .
<i>Arbitrated loop</i> (FC-AL)	Até 126 dispositivos-nó (NL_Ports) mais uma FL_Port, totalizando 127 endereços de anel, com arbitragem pelo direito de transmitir. Banda compartilhada: um dispositivo lento degrada todos, e a inserção de um nó reinicializa o anel (LIP), causando pausas de E/S. Foi a topologia dos <i>hubs</i> FC dos anos 1990 e das gavetas de disco FC-AL. Está obsoleta — não há produto novo de SAN sendo construído sobre ela.
<i>Switched fabric</i> (FC-SW)	É a topologia real de qualquer SAN FC moderna , e o “FC <i>Switch</i> ” do enunciado. Um ou mais <i>switches</i> formam uma <i>fabric</i> com comutação por <i>cut-through</i> , banda dedicada por porta, roteamento por FSPF, zoneamento, e escala de milhões de endereços (24 bits). <i>Switches</i> interligam-se por ISLs (<i>Inter-Switch Links</i>), que podem ser agregados em <i>trunks</i> . A prática padrão é montar duas <i>fabric</i>s fisicamente independentes (“SAN A” e “SAN B”), com o servidor tendo uma HBA em cada, de modo que nenhuma falha ou erro de configuração de uma <i>fabric</i> derrube o acesso ao dado.

Ordem de grandeza da latência de um salto FC

O *product brief* do *switch* Brocade G710 (Gen 7, 64GFC) declara: “a latência para portas comutadas localmente é de **460 ns** (incluindo FEC)” [15]. **Proveniência:** *datasheet* de fabricante, modelo identificado, condição declarada (portas comutadas localmente, com FEC).

Derivação nossa: um caminho típico servidor → *switch* → *array* atravessa 1 a 2 saltos, somando 0,46 a 0,92 μ s. Comparado com a latência de uma leitura aleatória em SSD NVMe — que Silberschatz situa em “20 a 100 microssegundos” [2] — a rede FC responde por **menos de 5%** do tempo total do acesso. Este é o argumento quantitativo central a favor da SAN: *a rede deixou de ser o gargalo*. Não é um argumento a favor de gastar mais em FC; é a explicação de por que a alternativa Ethernet passou a ser viável.

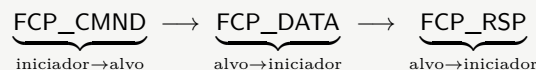
4.4 Protocolo SAN I — FCP sobre *Fibre Channel*

O **FCP** (*Fibre Channel Protocol*) é o mapeamento de camada FC-4 que transporta comandos, dados e respostas SCSI-3 sobre quadros FC. É o que se quer dizer, na prática, quando se diz “SAN FC”.

Como funciona, quadro a quadro — uma operação de E/S completa

Toda operação FCP é uma **troca** (*exchange*), composta de sequências de quadros. O iniciador abre a troca; o alvo a encerra.

Leitura de um bloco:



O FCP_CMND carrega o CDB (*Command Descriptor Block*) SCSI e o LUN alvo; o FCP_DATA traz a carga útil, em tantos quadros quantos forem necessários (a carga máxima por quadro é de 2.112 *bytes*); o FCP_RSP devolve o *status* SCSI e fecha a troca.

Escrita acrescenta um passo de controle de fluxo em nível de comando:



O FCP_XFER_RDY é o alvo informando *quantos bytes* está pronto para receber e a partir de que deslocamento. **É por isso que uma escrita FC custa uma ida e volta a mais que uma leitura** — fato que importa diretamente para a latência de *fsync* do *log* de transações discutida na Seção 2.2.

Duas camadas de controle de fluxo, que não se confundem. O XFER_RDY opera por *comando*, na camada FC-4; o **BB_Credit** opera por *quadro*, na camada FC-2, entre portas adjacentes. São independentes, e o congestionamento de uma não é visível na outra.

Dimensionamento por créditos de *buffer* — derivação nossa

Elmasri e Navathe citam “até 10 km de separação” como vantagem da SAN [1]. Quantos créditos isso exige? A propagação em fibra é de $\approx 5 \mu\text{s}/\text{km}$ (Seção 4.6), logo 10 km = 50 μs por sentido. A 32GFC, isto é, 3.200 MB/s por direção:

$$50 \times 10^{-6} \text{ s} \times 3,2 \times 10^9 \text{ B/s} = 160 \text{ KB em voo} \Rightarrow \frac{160 \times 10^3}{2.112} \approx \mathbf{76} \text{ quadros.}$$

Contando o retorno do crédito, o enlace precisa de aproximadamente **150 BB_Credits** para permanecer cheio a 10 km. Os **2.000 buffers** do Brocade G710 [15] estão, portanto, mais de uma ordem de grandeza acima do necessário para essa distância — e é isso que permite às portas de extensão sustentarem enlaces de dezenas a centenas de quilômetros sem ociosidade por falta de crédito. **Quando os créditos acabam, o enlace não descarta: ele para** — daí o fenômeno de *slow drain*, em que um único dispositivo lento congela tráfego alheio que compartilhe o caminho.

Suas características determinantes:

- **Rede sem perdas** por créditos de *buffer* (FC-2), o que dispensa TCP e sua sobrecarga de retransmissão, janelas e controle de congestionamento.
- **Rede fisicamente separada** do tráfego de aplicação, o que elimina interferência e simplifica enormemente o diagnóstico de desempenho.
- **Offload completo em hardware**: a HBA processa a pilha FC inteira, sem consumir CPU do servidor.
- **Custo e especialização**: exige HBAs, *switches*, transceptores e cabeamento dedicados, além de pessoal que saiba operar zoneamento e *fabrics* redundantes.

4.5 Protocolo SAN II — iSCSI

O que é. O iSCSI (*Internet SCSI*) encapsula comandos SCSI dentro de TCP/IP, permitindo montar uma SAN de bloco sobre Ethernet comum. A norma vigente é a **RFC 7143** (2014), que consolidou e obsoletou a **RFC 3720** (2004), originalmente citada em quase toda a literatura [5].

Correção de referência

Grande parte do material didático — inclusive resumos gerados por IA — ainda cita a RFC 3720 como “a norma do iSCSI”. Ela foi obsoletada pela **RFC 7143** em abril de 2014 [5]. Citar a 3720 hoje é apontar para documento superado.

Como funciona. Elmasri e Navathe descrevem o fluxo com clareza: “quando um SGBD precisa acessar dados, o sistema operacional gera os comandos SCSI apropriados e a requisição de dados, que então passam por procedimentos de encapsulamento e, se necessário, de criptografia. Um cabeçalho de pacote é adicionado antes que os pacotes IP resultantes sejam transmitidos por uma conexão Ethernet” [1]. Os elementos:

- **Iniciador** (o servidor) e **alvo** (o *array*), identificados por **IQN** (*iSCSI Qualified Name*), no formato `iqn.2026-09.br.ufrj.dcc:servidor01`.
- Transporte em **TCP**, **porta 3260**.
- Autenticação por **CHAP**; confidencialidade opcional por IPsec.
- Descoberta de alvos por **SENDTARGETS** ou por **iSNS**.

Mecanismo: os três pontos em que o iSCSI difere do FCP

(1) **R2T (*Ready to Transfer*)**. É o equivalente iSCSI do FCP_XFER_RDY: o alvo autoriza o envio dos dados de escrita. Duas otimizações reduzem idas e voltas — *ImmediateData* (dados de escrita viajam junto com o próprio comando) e *InitialR2T=No* (o iniciador envia uma primeira rajada sem esperar autorização). Ambas são negociadas no *login*, e desligá-las por engano custa uma ida e volta por escrita.

(2) **Digests**. O iSCSI oferece CRC-32C opcional de cabeçalho e de dados (*HeaderDigest/DataDigest*). O FC tem CRC obrigatório no quadro; o TCP tem apenas um *checksum* de 16 bits, notoriamente fraco. Habilitar *digest* custa CPU — e é justamente esse custo que uma HBA iSCSI ou um TOE assume.

(3) **Múltiplos caminhos: MC/S × MPIO**. *Multiple Connections per Session* (MC/S) agrega várias conexões TCP dentro de uma sessão iSCSI; o MPIO agrega sessões na camada de bloco do sistema operacional. Não são equivalentes: MC/S é do protocolo, MPIO é do *host* — e a recomendação usual dos fornecedores é MPIO, porque é ele que integra com ALUA e com as políticas de *failover* do sistema operacional (Seção 4.8).

Vantagens. Elmasri e Navathe são precisos: “a principal vantagem do iSCSI é que ele não requer o cabeamento especial necessário ao *Fibre Channel* e pode operar a distâncias maiores usando a infraestrutura de rede existente” [1]. E registram o efeito de mercado: “o armazenamento iSCSI impactou principalmente empresas de pequeno e médio porte, pela combinação de simplicidade, baixo custo e funcionalidade”, permitindo-lhes “não ter de aprender os detalhes da tecnologia *Fibre Channel*”; já as “implementações de iSCSI nos *data centers* de grandes empresas são lentas em desenvolvimento devido ao investimento prévio em SANs baseadas em *Fibre Channel*” [1].

Desvantagens, e o que mudou desde a edição do livro. O custo do iSCSI é a pilha TCP/IP: processamento no *host*, controle de congestionamento e perda de quadros sob congestão. As mitigações modernas são três, e nenhuma aparece no livro:

- **TOE** (*TCP Offload Engine*) ou HBA iSCSI dedicada, movendo a pilha para o adaptador;
- **iSER** (*iSCSI Extensions for RDMA*), que substitui a movimentação de dados do iSCSI por RDMA;
- **DCB** no *switch* Ethernet, dando ao tráfego de armazenamento uma classe sem perdas — a mesma tecnologia que o FCoE exige (Seção 4.7).

4.6 Protocolo SAN III — FCIP

O que é. O FCIP (*Fibre Channel over TCP/IP*) é padronizado pela **RFC 3821** (2004) [6]. Ele não é um substituto do FC: é um **túnel**. Elmasri e Navathe o definem exatamente assim: “o outro método, *Fibre Channel over IP* (FCIP), traduz códigos de controle e dados de *Fibre Channel* em pacotes IP para transmissão entre redes de área de armazenamento *Fibre Channel* geograficamente distantes. Este protocolo, conhecido também como **tunelamento *Fibre Channel*** ou **tunelamento de armazenamento**, só pode ser usado em conjunto com tecnologia *Fibre Channel*, ao passo que o iSCSI pode rodar sobre redes Ethernet existentes” [1].

A diferença conceitual entre iSCSI e FCIP, que é fácil de confundir

iSCSI substitui o FC. O servidor não tem HBA de *Fibre Channel*; ele fala SCSI sobre TCP/IP de ponta a ponta. Não existe *fabric* FC em lugar nenhum.

FCIP preserva o FC. As duas pontas são *fabrics* FC completas, com HBAs, WWNs e zoneamento. O FCIP apenas carrega os quadros FC de uma para a outra através da WAN IP, e as duas *fabrics* **se fundem em uma só** do ponto de vista do FC. Daí o principal risco operacional do FCIP: um evento de reconfiguração de *fabric* num sítio se propaga ao outro. Por isso os produtos oferecem *FC routing* / *IVR* para isolar as *fabrics* mantendo o acesso entre elas.

Caso de uso real, e único. Replicação síncrona ou assíncrona entre sítios para recuperação de desastre. Elmasri e Navathe já registram isso entre as vantagens da SAN: “replicação de dados de alta velocidade entre múltiplos sistemas de armazenamento. Tecnologias típicas usam replicação síncrona para local e replicação assíncrona para soluções de recuperação de desastres (DR)” [1].

Por que a distância decide entre síncrono e assíncrono — derivação nossa

A velocidade da luz em fibra óptica é aproximadamente 2×10^8 m/s (índice de refração $\approx 1,5$). Uma transação com replicação *síncrona* só confirma o *commit* depois que o sítio remoto reconhece a escrita, o que exige uma ida e volta:

$$t_{\text{RTT}} = \frac{2d}{2 \times 10^8 \text{ m/s}} \Rightarrow d = 100 \text{ km} \Rightarrow t_{\text{RTT}} = \frac{2 \times 10^5}{2 \times 10^8} = 1,0 \text{ ms.}$$

São **1 milissegundo por 100 km, só de propagação**, antes de somar o tempo de comutação, serialização e processamento nas duas pontas. Para uma carga OLTP cujo *commit* local custa $\approx 100 \mu\text{s}$, replicar sincronamente a 100 km multiplica a latência de *commit* por mais de dez. **Esta é a razão física** pela qual a replicação síncrona é restrita a *campus* e área metropolitana, e a replicação intercontinental sobre FCIP é obrigatoriamente assíncrona — com a consequência de aceitar um RPO (*Recovery Point Objective*) maior que zero. **Categoria:** derivação nossa a partir de constante física.

4.7 Protocolo SAN IV — FCoE

O que é. O FCoE (*Fibre Channel over Ethernet*) encapsula quadros FC *inteiros* diretamente em quadros Ethernet, **sem TCP e sem IP**. Foi padronizado pelo T11 em **FC-BB-5**, publicado como **ANSI/INCITS 462-2010** [7]. A versão seguinte, FC-BB-6, acrescentou o modo VN2VN (comunicação direta entre nós sem *switch* FCF).

Precisando o Elmasri & Navathe sobre FCoE — com o parágrafo inteiro à vista

Primeiro, a citação completa, porque recortá-la mudaria o julgamento. O livro escreve: “a ideia mais recente a entrar na corrida do armazenamento IP corporativo é o *Fibre Channel over Ethernet* (FCoE), *que pode ser pensado como iSCSI sem o IP*. Ele usa muitos elementos de SCSI e FC (assim como o iSCSI), mas não inclui componentes TCP/IP. [...] Ele tira proveito de uma tecnologia Ethernet confiável que usa *buffering* e **controle de fluxo fim-a-fim** para evitar pacotes descartados” [1].

O que o livro acerta, e que é preciso reconhecer. A frase de abertura é uma analogia didática, não uma definição — e o próprio parágrafo já a qualifica em dois pontos: diz que o FCoE *não* inclui TCP/IP, e diz que ele depende de Ethernet confiável com controle de fluxo. Uma crítica que ignorasse essas duas frases estaria atacando um espantalho. **Registramos que a primeira versão deste relatório fazia exatamente isso**, apresentando como “três razões” pelas quais o livro estaria errado dois pontos que o livro já cobre (Seção 13.6).

Restam duas imprecisões reais, e são específicas:

(1) **O controle de fluxo do FCoE não é “fim-a-fim”, é enlace a enlace.** O mecanismo é o IEEE 802.1Qbb, *Priority-based Flow Control* [8]: o quadro PAUSE atua entre *dois vizinhos adjacentes*, por prioridade, e cada salto exerce a pressão sobre o salto anterior. Não há realimentação entre origem e destino como há no TCP. A distinção não é acadêmica: é ela que produz o *congestion spreading* característico de redes FCoE mal dimensionadas, em que um alvo lento propaga pausas para trás e degrada tráfego não relacionado que compartilhe o caminho. Dizer “fim-a-fim” sugere um mecanismo que o FCoE não tem.

(2) **O FCoE não é roteável em camada 3; o iSCSI é.** Esta é a diferença que a analogia apaga e a de maior consequência prática. O FCoE é um *EtherType* próprio (0x8906) encapsulado diretamente em quadro Ethernet: não há cabeçalho IP, e portanto **não há o que rotear**. O FCoE vive dentro de um domínio de camada 2. O iSCSI, por rodar sobre TCP/IP, atravessa LAN, WAN e a Internet — vantagem que o próprio livro destaca ao tratar do iSCSI [1].

Consequência prática: um projetista que leia “FCoE é iSCSI sem o IP” e conclua que pode usá-lo entre dois *data centers*, como faria com iSCSI, vai descobrir que não pode. Para atravessar a WAN, os caminhos são FCIP (túnel entre *fabrics* FC) ou iSCSI.

Categoria: análise do grupo apoiada em norma (FC-BB-5 e IEEE 802.1Qbb), não citação. É juízo técnico, e o classificamos como tal.

Onde o FCoE se encaixa. A promessa do FCoE era a *convergência*: um único par de adaptadores (CNAs) e um único cabeamento carregando LAN e SAN, reduzindo adaptadores, cabos, portas e consumo. Elmasri e Navathe registram a adoção: “o FCoE foi produtizado com sucesso pela CISCO (denominado *Data Center Ethernet*) e pela Brocade” [1]. Na prática, a convergência prevaleceu *dentro do rack* (*blade chassis* e sistemas convergentes, onde o domínio de camada 2 é pequeno e controlado) e não substituiu o FC no núcleo da SAN nem deslocou o iSCSI nas instalações menores. O FCoE ocupa hoje um nicho estreito entre dois vizinhos mais simples.

4.8 Dois mecanismos que a SAN pressupõe: *multipath* e RAID

O requisito 5 da Seção 2.2 — comportamento determinado sob indisponibilidade transitória — não é atendido pelo protocolo sozinho. Quem o atende são dois mecanismos que ficam fora da discussão

de protocolo e dentro da de arquitetura.

Multipath e ALUA. Montar duas *fabrics* independentes (Seção 4.3) só produz disponibilidade se houver, no servidor, uma camada que reconheça que os dois caminhos levam ao *mesmo* LUN e escolha entre eles: é o *multipath* (DM-Multipath no Linux, MPIO no Windows). Em *arrays* com controladoras assimétricas, o padrão **ALUA** (*Asymmetric Logical Unit Access*) permite ao alvo informar quais caminhos são *otimizados* e quais são apenas *disponíveis*, para que o *host* não force o *array* a atravessar o *interconnect* interno entre controladoras. **Consequência direta para o SGBD:** o que decide se uma falha de caminho é invisível ou fatal é o tempo de detecção e de reprova (`no_path_retry`, `fast_io_fail_tmo`). Configurá-lo como “enfileirar indefinidamente” congela a instância; como “falhar imediatamente”, aborta transações. É decisão de projeto, não padrão a aceitar — e é a resposta concreta ao requisito 5.

RAID e a penalidade de escrita, aplicada ao log. Silberschatz dedica a Seção 12.5 ao RAID e é explícito quanto ao custo do nível 5: “o RAID nível 5 tem uma sobrecarga significativa para escritas aleatórias, já que uma única escrita aleatória de bloco requer 2 leituras de bloco (para obter os valores antigos do bloco e do bloco de paridade) e 2 escritas de bloco” [2]. E conclui, no mesmo trecho, que “o RAID nível 1 é popular para aplicações como o armazenamento de arquivos de *log* num sistema de banco de dados, já que oferece o melhor desempenho de escrita” [2].

A penalidade de escrita aplicada ao WAL — derivação nossa

Custo, em operações físicas, de uma escrita aleatória de um bloco:

RAID 1: 2 **RAID 5:** 4 (2 leituras + 2 escritas) **RAID 6:** 6 (3 + 3)

O *log* de transações é escrito sequencialmente e sincronizado a cada *commit*. Sob RAID 5, toda descarga de *log* que não preencha uma faixa completa paga a penalidade de 4× — e é justamente a taxa de *commits* que ela limita. **Daí a regra prática, a mesma há trinta anos, e que o próprio livro enuncia:** *log* em RAID 1 ou 10, dados em RAID 5 ou 6. É também a razão pela qual a política de AST para o volume de *redo* deve ser fixada, e não deixada ao critério estatístico do *array* (Seção 6).

4.9 Nota de fronteira: NVMe-oF

Uma observação de honestidade técnica: nenhum dos três livros da disciplina cobre o **NVMe-oF** (*NVMe over Fabrics*), porque ele amadureceu depois das edições adotadas. Ele é relevante para a discussão porque *substitui o SCSI*, e não o meio de transporte. O SCSI foi projetado nos anos 1980 para discos mecânicos, com uma única fila de comandos; o NVMe foi projetado para *flash*, com até 65.535 filas. Rodar NVMe sobre a mesma rede FC (**FC-NVMe**), sobre TCP (**NVMe/TCP**) ou sobre RDMA (**NVMe/RoCE**) elimina a tradução para comandos SCSI e a serialização que ela impõe. Registramos aqui porque o quadro “FC × iSCSI × FCoE” do enunciado descreve corretamente o estado da arte dos livros, mas a fronteira de 2026 é “SCSI × NVMe” tanto quanto “FC × Ethernet”. O aprofundamento está fora do escopo desta entrega.

4.10 Vantagens e desvantagens da SAN

Tabela 8: SAN: vantagens e desvantagens sob a ótica de um SBD.

Vantagens	Desvantagens
Semântica de bloco cru: o SGBD mantém controle total sobre alocação, <i>cache</i> e ordenação de escrita.	Custo de aquisição e de operação: HBAs, <i>switches</i> , transceptores, cabeamento e duas <i>fabrics</i> redundantes.
Rede dedicada, sem interferência do tráfego de aplicação.	Exige competência especializada: zoneamento, <i>LUN masking</i> , <i>multipath</i> , diagnóstico de <i>fabric</i> .
“Conectividade flexível muitos-para-muitos entre servidores e dispositivos de armazenamento” [1].	Compartilhar um LUN entre servidores exige sistema de arquivos de <i>cluster</i> ou gerenciador de volume ciente de <i>cluster</i> .
“Até 10 km de separação entre servidor e sistema de armazenamento com cabos de fibra óptica adequados” [1]; além disso, por FCIP.	Encapsulamento SCSI legado herda um modelo de fila projetado para disco mecânico (ver NVMe-oF).
“Capacidades de isolamento melhores, permitindo adição não disruptiva de novos periféricos e servidores” [1].	Elmasri e Navathe registram os problemas remanescentes: “combinar opções de armazenamento de múltiplos fornecedores e lidar com padrões em evolução de <i>software</i> e <i>hardware</i> de gerenciamento” [1].
Latência de rede desprezível diante da latência da mídia (<5% num acesso NVMe; derivação da Seção 4.3).	<i>Boot</i> e recuperação dependem da <i>fabric</i> : um erro de zoneamento pode tornar o dado inacessível instantaneamente para todos os servidores.

5 Quadro comparativo consolidado

Antes da grade, uma figura. Boa parte da confusão entre os cinco protocolos — e quase toda a confusão entre iSCSI, FCIP e FCoE — desaparece quando se vê *o que* cada um coloca dentro de *o quê*. A Figura 2 mostra as cinco pilhas lado a lado.

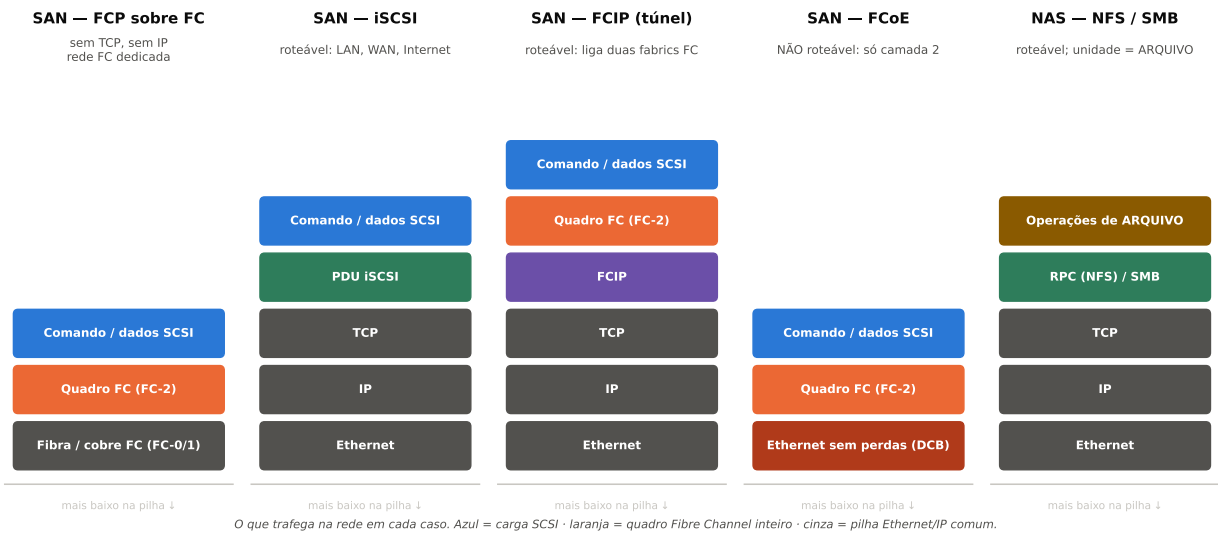


Figura 2: O que trafega na rede em cada arquitetura. Três leituras que a figura torna imediatas: (i) **iSCSI e FCoE não são variantes um do outro** — o iSCSI carrega *comandos SCSI* sobre TCP/IP, enquanto o FCoE carrega o *quadro FC inteiro* sobre Ethernet; (ii) **FCIP e FCoE carregam a mesma coisa** (o quadro FC), e diferem apenas em sobre o quê — daí um ser roteável e o outro não; (iii) **o NAS é o único em que a unidade não é o bloco**: no topo da pilha há operações de arquivo, e não comandos SCSI. Essa última diferença é a da Seção 3, e é a que decide tudo o mais.

A Tabela 9 reúne, em uma única grade, **todos** os protocolos listados pelo enunciado, cada um como uma linha, com sua norma de origem e as propriedades que decidem sua aplicabilidade a um SBD.

Tabela 9: Todos os protocolos exigidos pelo enunciado, com norma de origem e propriedades decisivas. “Unidade” é o que efetivamente trafega na rede.

Protocolo	Família	Norma / origem	Unidade trans- portada	Transporte	Roteável em L3?	Estado em 2026	Aplicabilidade a SGBD
SMB/CIFS	NAS	Especificação aberta Microsoft [MS-SMB2]; “CIFS” = dialeto SMB 1.0	Operações de arquivo	TCP porta 445 (antes NetBIOS 139)	Sim	SMB 3.1.1 corrente; SMB 1 não instalado por padrão desde o Windows 10 <i>Fall Creators Update</i> [11]	Suportado para SQL Server 2012+ (bancos de sistema e de usuário), exigindo SMB 3 <i>transparent failover</i> para carga crítica; FILESTREAM não suportado [18]
NFS	NAS	IETF: RFC 1094 (v2), RFC 1813 (v3), RFC 7530 (v4.0), RFC 8881 (v4.1), RFC 7862 (v4.2)	Operações de arquivo (RPC)	TCP porta 2049 (v4); v3 usa port-mapper + NLM	Sim	v4.1 é a norma vigente; pNFS separa metadados de dados [4]	Suportado com ressalvas: exige 0_DIRECT ou cliente próprio por causa da coerência <i>close-to-open</i> [12]. Oracle implementa cliente próprio (dNFS) [17]
AFP	NAS	Proprietário Apple (<i>AppleTalk Filing Protocol</i> , 1985–88)	Operações de arquivo	TCP porta 548	Sim	Encerrado: servidor removido no macOS 11 (2020); cliente depreciado no macOS 15.5 (mai./2025) [20] e ausente nos <i>betas</i> do macOS 27 (jun./2026) [28]	Nenhuma. Sem travamento em intervalo de <i>bytes</i> adequado; nenhum SGBD de porte o suporta. Migrar para SMB 3
FCP sobre FC	SAN	T11/INCITS (FC-FS, FC-PI); FCP = mapeamento FC-4 de SCSI-3	Comandos e dados SCSI em quadros FC	<i>Fibre Channel</i> (rede sem perdas por BB_Credit)	Não (rede FC própria, endereço de 24 bits)	Padrão do <i>data center</i> ; 64GFC corrente, 128GFC (Gen 8) em implantação [14]	Referência. Bloco cru, rede dedicada, latência de rede <5% do acesso NVMe. Base do Oracle ASM e de <i>clusters</i> com armazenamento compartilhado
FC Switch (<i>switched fabric</i> , FC-SW)	SAN	T11/INCITS — é uma topologia , não um protocolo	(transporta FCP)	<i>Fibre Channel</i> comutado, roteamento FSPF	Não	Única topologia FC em uso; ponto-a-ponto residual, FC-AL obsoleto	Obrigatória em produção: banda dedicada por porta, zoneamento, e duas <i>fabrics</i> independentes (SAN A / SAN B)
iSCSI	SAN	IETF RFC 7143 (2014), que obsoleta a RFC 3720 [5]	Comandos SCSI em TCP	TCP porta 3260 sobre Ethernet/IP	Sim — LAN, WAN e Internet [1]	Consolidado em pequeno e médio porte; acelerado por TOE, iSER e DCB	Muito adequada quando a rede é dedicada ou segregada. Entrega bloco cru sem o custo de <i>fabric</i> FC
FCIP	SAN	IETF RFC 3821 (2004) [6]	Quadros FC inteiros em TCP/IP	TCP/IP sobre WAN	Sim	Vivo, com propósito único	Não é caminho de dado primário. É túnel entre duas <i>fabrics</i> FC para replicação entre sítios. A distância impõe assincronia (ver derivação da Seção 4.6)

Protocolo	Família	Norma / origem	Unidade trans- portada	Transporte	Roteável em L3?	Estado em 2026	Aplicabilidade a SGBD
FCoE	SAN	T11 FC-BB-5 = ANSI/INCITS 462-2010; FC-BB-6 (VN2VN) [7]	Quadros FC inteiros em quadros Ethernet, sem TCP e sem IP	Ethernet sem per- das obrigatório: DCB (802.1Qbb PFC, 802.1Qaz ETS, DCBX) + <i>baby jumbo</i>	Não — “não é roteável na camada IP e não funcionará através de redes IP roteadas como a Internet” [7]	Nicho: convergên- cia dentro do rack (<i>blade</i> , sistemas convergentes)	Viável mas raro. Perdeu para o iSCSI embaixo e para o FC em cima. Não atravessa <i>data centers</i>
(fronteira) NVMe-oF	SAN	NVM Express (FC- NVMe, NVMe/TCP, NVMe/RoCE)	Comandos NVMe	FC, TCP ou RDMA	Depende do trans- porte	Em adoção; subs- titui o <i>SCSI</i> , não o meio	Remove a fila única do SCSI. Fora do escopo dos livros da disciplina; registrado por completude

5.1 NAS × SAN: as dimensões que realmente decidem

Tabela 10: Comparação estrutural NAS × SAN.

Dimensão	NAS	SAN
Unidade de abstração	Arquivo	Bloco (LUN)
Dono do sistema de arquivos	O dispositivo de armazenamento	O servidor
Rede	LAN Ethernet/IP compartilhada	Rede dedicada (FC) ou Ethernet segregada
Protocolos	SMB/CIFS, NFS, AFP	FCP/FC, iSCSI, FCIP, FCoE
Compartilhamento entre servidores	Nativo; o dispositivo arbitra	Exige sistema de arquivos de <i>cluster</i> ou LVM ciente de <i>cluster</i>
Travamento	No protocolo (NLM no NFSv3; integrado a partir do NFSv4; <i>oplocks/leases</i> no SMB)	No servidor; a SAN não sabe o que é um arquivo
Coerência de <i>cache</i>	Fraca por padrão (<i>close-to-open</i>) [12]	Não se aplica: o <i>cache</i> é do servidor
Custo relativo	Menor (usa infraestrutura existente)	Maior (HBA, <i>switch</i> , cabeamento, pessoal)
Distância	Limitada pela LAN	10 km em fibra [1]; ilimitada por FCIP
Caso de uso típico em SBD	<i>Data warehouse</i> , arquivos de <i>backup</i> , ambientes de desenvolvimento/homologação, bancos de médio porte com dNFS	OLTP crítico, <i>clusters</i> com armazenamento compartilhado, cargas com requisito de latência de <i>commit</i>

A convergência que os livros já anunciavam

Silberschatz registra: “sistemas SAN e NAS modernos suportam o uso de uma combinação de discos magnéticos e SSDs, e podem ser configurados para usar os SSDs como *cache* para dados que residem em discos magnéticos” [2]. Na prática de 2026, quase todo *array* corporativo é **unificado**: o mesmo equipamento apresenta LUNs por FC ou iSCSI e compartilhamentos por NFS e SMB, sobre o mesmo *pool* de mídia. A pergunta “comprar NAS ou SAN?” virou, em grande medida, “qual protocolo apresentar para cada carga?” — o que é uma pergunta melhor, porque admite respostas diferentes para o *tablespace* e para a área de *dump*.

6 AST — Automated Storage Tiering

6.1 O que é e por que existe

Elmasri e Navathe definem: “outra tendência em armazenamento é o *automated storage tiering* (AST), que move automaticamente dados entre diferentes tipos de armazenamento, como SATA, SAS e *solid-state drives* (SSDs), dependendo da necessidade. O administrador de armazenamento pode configurar uma política de *tiering* na qual dados usados com menos frequência são movidos para *drives* SATA mais lentos e baratos, e dados usados com mais frequência são movidos para *solid-state drives*” [1]. E citam a implementação de referência: “a EMC tem uma implementação dessa tecnologia chamada FAST (*fully automated storage tiering*) que faz monitoramento contínuo da atividade de dados e toma ações para mover os dados para o *tier* apropriado com base na política” [1].

O AST existe porque a hierarquia de armazenamento de Silberschatz (Seção 2.1) é uma *escada de preço por byte*, e a distribuição de acesso a dados é altamente enviesada: em praticamente qualquer banco de dados corporativo, uma fração pequena dos blocos concentra a maioria esmagadora dos acessos. Comprar *flash* para o banco inteiro é pagar pelo pior caso em 100% da capacidade.

6.2 Como funciona, concretamente

Tomamos como exemplo documentado o Dell EMC Unity FAST VP, cujo *white paper* técnico publica os parâmetros exatos [16]. **Proveniência:** documentação de fabricante, produto identificado.

Tabela 11: Parâmetros do FAST VP (Dell EMC Unity). Fonte: *white paper* H15086 [16].

Parâmetro	Valor documentado
Granularidade de relocação	Fatias (<i>slices</i>) de 256 MB: “em um <i>pool</i> multicamada, os dados dos recursos de armazenamento são espalhados pelos <i>tiers</i> pelo FAST VP em fatias de 256 MB”
<i>Tiers</i> definidos	<i>Extreme Performance (flash)</i> ; <i>Performance</i> (SAS 10K/15K rpm); <i>Capacity</i> (NL-SAS 7,2K rpm)
Frequência de análise	“Uma vez por hora, o FAST VP analisa os dados coletados e classifica cada fatia com base na <i>temperatura</i> de cada fatia”
Janela de relocação	Agendada e configurável; padrão diário, das 17h à 1h
Políticas	<i>Highest Available Tier</i> ; <i>Auto-Tier</i> ; <i>Start High then Auto-Tier</i> (padrão recomendado); <i>Lowest Available Tier</i>

O mecanismo é, portanto: (i) contadores de acesso por fatia; (ii) uma métrica agregada de “temperatura”; (iii) uma classificação horária; (iv) e a movimentação física das fatias entre *tiers* durante uma janela em que o impacto sobre a carga de produção é aceitável.

6.3 A tensão que ninguém menciona: AST versus *buffer pool*

Aqui está o ponto analítico deste trabalho sobre AST. **Ressalva de alcance, antes de tudo:** não afirmamos originalidade absoluta — as próprias recomendações de fabricante (fixar a política do volume de *redo*, por exemplo) pressupõem essa tensão sem nomeá-la. O que verificamos é mais estreito e é o que afirmamos: **nenhum dos três livros-texto da disciplina faz a ligação entre a Seção do *buffer manager* e a Seção do *tiering***, embora ambas estejam no mesmo capítulo do Elmasri. A contribuição é a *articulação*, não a observação isolada.

O SGBD já faz *tiering* — e faz melhor, mais rápido e com mais informação

Elmasri e Navathe dedicam a Seção 16.3.1 inteira ao *buffer manager*, que “responde a requisições por dados e decide qual *buffer* usar e quais páginas substituir”, com políticas LRU, *clock* e FIFO, além de *pin-count* e *dirty bit* [1]. Isso é *tiering* — entre a memória principal e o armazenamento secundário. E ele opera com três vantagens decisivas sobre o AST do *array*:

(1) **Granularidade.** O *buffer manager* decide por **página de 8 ou 16 KiB**. O FAST VP decide por **fatia de 256 MB** [16]. **Derivação nossa, com a convenção declarada:** lendo os “256 MB” da Dell como 256 MiB,

$$\frac{256 \text{ MiB}}{16 \text{ KiB}} = 16.384\times \quad \text{e} \quad \frac{256 \text{ MiB}}{8 \text{ KiB}} = 32.768\times.$$

O fator depende do SGBD, e por isso publicamos os dois: 16.384× para a página de 16 KiB do InnoDB e 32.768× para a de 8 KiB do PostgreSQL e do Oracle. **Usamos o menor dos dois em todo o restante do texto**, por ser o mais conservador para o nosso próprio argumento. (Se os “256 MB” forem lidos como decimais, as razões caem para 15.625 e 31.250; adotamos a leitura binária, usual para tamanhos de bloco.) De todo modo a ordem de grandeza é a mesma: se dentro de uma fatia houver 1 MB quente e 255 MB frios, o AST promove os 256 MB inteiros.

(2) **Latência de reação.** O *buffer manager* reage **ao acesso**. O AST reage em uma janela de **horas** [16].

(3) **Informação semântica.** O *buffer manager* sabe que aquela página é o nó raiz de um índice B⁺-tree e que ela será acessada em toda busca. O *array* vê apenas *offsets* de LBA.

Disso decorrem três consequências práticas que devem constar de qualquer recomendação de AST para SBD:

1. **O AST enxerga a carga *filtrada* pelo *buffer pool*.** O que chega ao *array* é o *resíduo* que o *cache* do SGBD não absorveu. Um bloco genuinamente quente pode nunca aparecer como quente para o AST, porque ele mora permanentemente no *buffer pool* e quase nunca é relido do disco. **O AST pode, portanto, classificar como frio exatamente o dado mais quente do banco.**
2. **Cargas OLTP com dados quentes móveis derrotam o AST.** Se o conjunto quente muda mais rápido que a janela de relocação (o caso típico de dados particionados por data, em que a partição quente muda diariamente), o AST passa a maior parte do tempo movendo dados que acabaram de esfriar. A política *Start High then Auto-Tier*, que a Dell recomenda como

padrão [16], existe precisamente para mitigar isso: escreve tudo no *tier* mais rápido primeiro e só depois esfria o que não for acessado.

3. **Caching é mais adequado que tiering para banco de dados.** A distinção é importante e frequentemente confundida: no *tiering*, o dado *muda de lugar* (só existe uma cópia, num *tier*); no *caching*, o dado é *copiado* para a camada rápida e o original permanece. *Caching* reage em segundos e com granularidade fina, sem custo de migração; *tiering* reage em horas com granularidade grossa, mas converte capacidade cara em capacidade barata de forma permanente. Para OLTP, *caching* é quase sempre a resposta certa. Não por acaso, Silberschatz descreve exatamente essa configuração: “sistemas SAN e NAS modernos [...] podem ser configurados para usar os SSDs como **cache** para dados que residem em discos magnéticos” [2].

6.4 Recomendação sobre AST

- **Habilitar** em *data warehouses* e ambientes com particionamento por data e grande volume histórico frio: o padrão de acesso é estável em escala de dias, que é a escala do AST.
- **Habilitar** em consolidação de muitos bancos pequenos num *pool* comum, em que o desbalanceamento de temperatura entre bancos é grande e persistente.
- **Evitar** (ou fixar em *Highest Available Tier*) para arquivos de *redo*/WAL, *tempdb* e índices críticos: são estruturas cujo desempenho não pode depender de uma decisão estatística tomada com atraso de horas.
- **Nunca** usar AST como substituto de dimensionamento. Se o conjunto de dados quente não cabe no *tier* rápido, o AST não cria capacidade — ele apenas redistribui a escassez, e acrescenta tráfego de migração à carga já saturada.

7 Object-Based Storage

7.1 O terceiro paradigma

Bloco e arquivo não são as duas únicas abstrações possíveis. Elmasri e Navathe apresentam a terceira: “sob esse esquema, os dados são gerenciados na forma de **objetos**, em vez de arquivos feitos de blocos. Os objetos carregam **metadados** que contêm propriedades que podem ser usadas para gerenciá-los. Cada objeto carrega um **identificador global único** que é usado para localizá-lo” [1].

Tabela 12: Os três paradigmas de armazenamento em rede.

	Bloco (SAN)	Arquivo (NAS)	Objeto
Unidade	Bloco de tamanho fixo	Arquivo em hierarquia de diretórios	Objeto com metadados e ID global
Endereçamento	LUN + LBA	Caminho hierárquico	Espaço de nomes plano (<i>bucket</i> + chave)
Interface	SCSI / NVMe	POSIX, SMB, NFS	HTTP REST (PUT, GET, DELETE)
Atualização parcial	Sim, qualquer bloco	Sim, qualquer <i>offset</i>	Não: o objeto é substituído inteiro
Escala típica	Terabytes a petabytes	Terabytes a petabytes	Exabytes
Metadados	Nenhum (o <i>array</i> não sabe o que guarda)	Fixos (dono, datas, permissões)	Arbitrários, definidos pela aplicação

7.2 Origem acadêmica

Elmasri e Navathe registram a genealogia corretamente: “o armazenamento por objetos tem suas origens em projetos de pesquisa na CMU (Gibson et al., 1996) sobre escalabilidade de *network attached storage* e no sistema *OceanStore* em UC Berkeley (Kubiatowicz et al., 2000), que tentava construir uma infraestrutura global sobre todas as formas de servidores confiáveis e não confiáveis para acesso contínuo a dados persistentes” [1]. Registram também a tentativa de padronização: “comandos de dispositivo de armazenamento baseado em objeto (OSDs) foram propostos como parte

do protocolo SCSI há muito tempo, mas não se tornaram produto comercial até que a Seagate adotou OSDs em sua plataforma *Kinetic Open Storage* [1].

A mesma ideia arquitetural aparece três vezes neste trabalho, com nomes diferentes

O que o NASD de Gibson *et al.* propôs em 1996 não foi “guardar objetos”. Foi **separar o caminho de metadados e autorização do caminho de dados**: o cliente pede ao servidor de metadados uma *capacidade* (um *token* criptográfico que autoriza um acesso específico) e depois fala *direto* com o dispositivo de armazenamento, sem que os dados passem pelo servidor. O gargalo deixa de ser o servidor de arquivos.

É exatamente a mesma ideia de:

- **pNFS** (NFSv4.1, Seção 3.4): o cliente obtém um *layout* do servidor de metadados e lê os dados diretamente dos servidores de dados [4];
- **NDMP** (Seção 8.3): o servidor de *backup* controla, mas os dados vão do NAS direto para a fita;
- **Amazon Aurora** (Seção 8.2): o motor manda *log* e a camada de armazenamento materializa as páginas por conta própria [24].

A observação que tiramos disso: a “terceira via” do armazenamento não é o objeto — é a *desagregação do plano de controle e do plano de dados*. O objeto é uma das formas que ela assumiu; o pNFS é outra, dentro do NAS; o Aurora é outra, dentro do SGBD. Ler os três como instâncias do mesmo movimento explica por que o objeto venceu onde venceu e por que não venceu em OLTP: o que ele desagrega bem é a leitura em massa, não a ordenação de escrita.

Nota de atualização sobre a Seagate Kinetic

O exemplo da Seagate Kinetic era corrente na edição de 2016 do livro. A plataforma **não obteve tração comercial**: a cobertura técnica de março de 2016 registra que a HGST “passou um ano tentando encontrar casos de uso que valessem a pena para *drives* Kinetic junto a seus clientes, e não achou nenhum”, e que o projeto poderia voltar a ser apenas uma iniciativa proprietária da Seagate [10]. Na mesma linha, a padronização OSD no T10 não se estabeleceu como caminho da indústria.

Declaração de lacuna: não localizamos anúncio formal de descontinuação da plataforma Kinetic. Dizer “foi descontinuada” seria afirmar mais do que a fonte sustenta; dizemos “não obteve tração comercial”, que é o que está documentado.

O que venceu foi outro caminho — e é justo dizer que o próprio livro já aponta para ele. Elmasri e Navathe escrevem, no mesmo trecho, que “o armazenamento por objetos é a escolha de muitas ofertas de nuvem, como o S3 da AWS e o Azure da Microsoft”, e que “o OpenStack Swift é um projeto de código aberto que permite usar HTTP GET e PUT para recuperar e armazenar objetos — essa é basicamente toda a API” [1]. **Não há, portanto, novidade nossa em dizer que o verbo HTTP venceu o comando SCSI: o livro registra o fato.** O que acrescentamos é apenas a consequência de dez anos depois — que a API do S3 especificamente virou a interface que os demais implementam por compatibilidade (Ceph RADOS Gateway, MinIO, *appliances* corporativos). **Categoria:** leitura de mercado nossa, apoiada na ubiquidade das implementações, *sem* fonte primária que a quantifique — e portanto apresentada como leitura, não como fato.

7.3 Propriedades: por que escala tanto

O armazenamento por objetos abre mão de coisas para ganhar escala. Cada renúncia é deliberada:

- **Abre mão da hierarquia** → o espaço de nomes plano é particionável por *hash* sem coordenação central, o que remove o gargalo do servidor de metadados.
- **Abre mão da atualização parcial** → objetos imutáveis eliminam *read-modify-write* distribuído, tornam a replicação trivialmente correta e permitem versionamento nativo.
- **Abre mão do POSIX** → sem *fsync*, sem travamento em intervalo de *bytes*, sem semântica de *rename* atômico; sobra HTTP, que atravessa qualquer rede.
- **Em troca, ganha durabilidade extrema por codificação de apagamento.** A AWS declara que o S3 é “projetado para durabilidade de **99,999999999% (11 noves)**” [22]. **Derivação nossa para dar escala a esse número:** com 11 noves, a expectativa é de perder 10^{-11} dos objetos por ano; para 10^7 objetos armazenados, o número esperado de perdas é $10^7 \times 10^{-11} = 10^{-4}$ por ano, isto é, uma perda esperada a cada 10.000 anos. **Ressalva importante de rótulo:** isso é durabilidade *projetada* (do inglês *designed for*), não medida — e não protege contra exclusão acidental nem contra *ransomware*, apenas contra falha de mídia.

Uma atualização essencial: o S3 não é mais eventualmente consistente

Durante 14 anos, a objeção padrão ao armazenamento por objetos para dados de banco era a **consistência eventual**: após uma escrita, uma leitura podia devolver a versão antiga. Isso mudou. Desde **dezembro de 2020**, a AWS documenta que o S3 fornece “consistência forte *read-after-write* automaticamente”, de modo que, “após uma escrita ou sobrescrita bem sucedida, requisições de leitura subsequentes recebem imediatamente a versão mais recente do objeto”, valendo também para operações de listagem [22]. **Qualquer texto que ainda afirme que o S3 é eventualmente consistente está desatualizado em mais de cinco anos** — e essa era uma das afirmações que a nossa própria primeira versão continha (ver Seção 13.4).

7.4 Por que ainda não serve como armazenamento primário de OLTP

Elmasri e Navathe encerram a seção com uma ressalva que continua correta: “como o armazenamento por objetos força o travamento a ocorrer no nível do objeto, não está claro quão adequado ele é para processamento concorrente de transações em sistemas orientados a transações de alta vazão. Portanto, ainda não é considerado viável para aplicações de banco de dados corporativas de *mainstream*” [1]. Detalhando as razões contra os cinco requisitos da Seção 2.2:

1. **Sem atualização parcial.** Alterar uma linha exigiria reescrever o objeto inteiro. Um *tablespace* de 100 GB como um objeto é inviável; fatiado em objetos de 8 KiB, perde-se toda a vantagem de escala e ganha-se uma requisição HTTP por página.
2. **Latência de uma ordem de grandeza acima.** Silberschatz já registrava: “armazenamento em nuvem tem latência muito alta, de dezenas a centenas de milissegundos, se os dados não estiverem colocalizados com o banco de dados, e portanto não é ideal como armazenamento subjacente para bancos de dados” [2].
3. **Sem fsync nem ordenação de escrita** entre objetos distintos: não há como exprimir “grave o *log* antes da página”.
4. **Granularidade de travamento incompatível** com concorrência transacional.

7.5 Onde ele venceu, e venceu completamente

- **Camada de armazenamento de *data warehouses* em nuvem.** A arquitetura de separação entre computação e armazenamento (Snowflake, Databricks, BigQuery) coloca os dados em objetos imutáveis em formato colunar (Parquet, ORC), com uma camada de metadados transacional por cima (Iceberg, Delta Lake). O padrão de acesso — escrever uma vez, ler muitas, varrer grandes extensões — é exatamente aquele em que o objeto é ótimo e a latência não importa.
- **Destino de *backup* e arquivamento** (Seção 8.3).
- **Repositórios de dados não estruturados.** Elmasri e Navathe citam Facebook, Spotify e Dropbox [1]; o caso do Dropbox é discutido na Seção 9.

8 Recomendação geral

Esta seção responde à pergunta central do enunciado: *quando optar por um ou pelo outro tipo de storage, considerando como o núcleo do SBD (Database Engine) utilizará a solução, tanto em nível secundário (online) quanto em nível terciário (offline).*

8.1 Princípio orientador**A regra de decisão, em uma frase**

Escolha pela unidade de abstração que o *Database Engine* precisa controlar, não pela velocidade do enlace. Se o motor precisa ser dono da ordenação de escrita, da atomicidade da página e do travamento — OLTP transacional, *cluster* com armazenamento compartilhado — ele precisa de **bloco**, e portanto de SAN. Se o motor pode delegar essas garantias porque a carga é de leitura, de varredura ou de ingestão em lote, então **arquivo** basta, e o NAS entrega o mesmo resultado por uma fração do custo.

O corolário quantitativo vem da derivação da Seção 4.3: a rede FC contribui com menos de 5% da latência de um acesso NVMe. **A rede deixou de ser o gargalo**, e portanto o argumento “FC é mais rápido” perdeu força; o que resta como diferença real é a *semântica*, não a velocidade.

8.2 Nível secundário (*storage online*)

É onde vivem os arquivos de dados, os índices e o *log* de transações — tudo que o motor acessa durante a execução normal. A recomendação é por **tipo de carga**, não por tipo de empresa.

Tabela 13: Recomendação para o nível secundário, por perfil de carga.

Perfil de carga	Recomendação	Justificativa
OLTP crítico, alta taxa de <i>commit</i>	SAN (FC ou iSCSI em rede dedicada)	A latência de fsync do <i>log</i> governa a vazão de transações. Bloco cru elimina a camada de sistema de arquivos remoto do caminho crítico e dá ao motor o controle da ordenação.
<i>Cluster</i> com armazenamento compartilhado (Oracle RAC)	SAN (bloco)	Vários nós escrevem nos mesmos blocos com coordenação própria do SGBD (Oracle ASM sobre LUNs). É a configuração de referência do fornecedor.
Banco relacional de médio porte, OLTP moderado	NAS ou SAN , indiferente	Com NFSv4.1 e O_DIRECT (ou dNFS [17]), a diferença é operacionalmente irrelevante. Decida pelo custo e pela competência da equipe.
<i>Data warehouse</i> , OLAP, varredura sequencial	NAS	O custo dominante é banda, não latência: um acesso já amortiza o custo de rede sobre megabytes. O NAS entrega a mesma banda por menos, e o compartilhamento nativo entre nós de processamento é uma vantagem, não um problema.
Desenvolvimento, homologação, muitas instâncias pequenas	NAS	Provisionamento simples, <i>clones</i> finos por instantâneo de arquivo, e nenhum requisito de latência.
Área de <i>dump</i> , <i>export</i> , <i>staging</i> de ETL	NAS	Semântica de arquivo é exatamente a abstração desejada; e escrever <i>dumps</i> num LUN dedicado é desperdício.
Banco em nuvem gerenciado	Bloco de rede (equivalente a SAN)	Volumes de bloco em nuvem (EBS e equivalentes) são SAN sob outro nome: iniciador no <i>host</i> , alvo remoto, semântica de bloco.

A observação que mais surpreende: o modelo pode ser abandonado por inteiro

Há um terceiro caminho, que não é NAS nem SAN, e que a arquitetura do **Amazon Aurora** exemplifica. Em vez de escolher entre bloco e arquivo, ela **redesenha a fronteira**: “as únicas escritas que cruzam a rede são registros de *redo log*. Nenhuma página é jamais escrita a partir da camada de banco de dados”, porque “o *log* é o banco de dados, e quaisquer páginas que o sistema de armazenamento materialize são simplesmente um *cache*” [24]. Os autores justificam a decisão exatamente com o argumento desta seção: “o gargalo se move para a rede entre a camada de banco de dados que requisita E/S e a camada de armazenamento” [24].

Resultado publicado. No experimento da Tabela 1 do artigo (SIGMOD 2017), o Aurora sustentou **27.378.000** transações contra **780.000** do MySQL espelhado sobre EBS — “35 vezes mais transações” — com **0,95** operações de E/S por transação contra **7,4**, que o artigo descreve como “7,7 vezes menos” [24]. **Proveniência:** medição publicada em artigo revisado por pares; os números são do experimento específico descrito no artigo e não devem ser generalizados como desempenho de produto.

Divergência que encontramos ao conferir a conta: $7,4 \div 0,95 = 7,79$, isto é, **7,8×** e não os **7,7×** que o texto do artigo enuncia. A diferença é de arredondamento e irrelevante para o argumento, mas registramos porque a regra deste trabalho é refazer toda conta citada — inclusive as de artigos revisados por pares.

Por que isso importa para a pergunta do enunciado: a resposta “NAS ou SAN?” pressupõe que o motor escreva páginas. Quando o motor deixa de escrever páginas e passa a escrever só *log*, a pergunta muda de objeto.

8.3 Nível terciário (*storage offline*)

Antes de comparar mídias: a pergunta que o enunciado faz é NAS × SAN

A Seção 2.1 estabeleceu que NAS e SAN são, ambos, tecnologias de nível *secundário*. Isso poderia sugerir que a pergunta “NAS ou SAN?” não se aplica ao nível terciário. **Aplica-se, e de três maneiras concretas:**

(1) A biblioteca de fitas é um dispositivo de SAN. A especificação do LTO-10 declara sua taxa comprimida “usando a interface *Fibre Channel* de 32 Gb” [21]. Os *drives* de fita são alvos FC (ou SAS) na mesma *fabric* dos *arrays*. A consequência de projeto é direta: **a janela de *backup* disputa a mesma *fabric* que a carga de produção**, e o zoneamento precisa separá-las. Um *backup* completo a 400 MB/s por *drive*, com oito *drives* em paralelo, são 3,2 GB/s sustentados atravessando a SAN — comparável à carga OLTP de pico de muitas instalações.

(2) O estágio intermediário é NAS. A prática dominante não escreve do banco direto para a fita: escreve primeiro em disco (*disk-to-disk-to-tape*), e esse estágio é quase sempre um compartilhamento NAS — semântica de arquivo é exatamente a abstração desejada para um conjunto de arquivos de *dump* nomeados por data. O NAS é, portanto, a **porta de entrada** do nível terciário.

(3) Fazer *backup* de um NAS exige um protocolo próprio: o NDMP. Um NAS corporativo não roda agente de *backup*; o NDMP (*Network Data Management Protocol*) separa o canal de *controle* (o servidor de *backup* orquestra) do canal de *dados* (o NAS escreve direto no *drive* de fita, sem que os dados atravessem o servidor de *backup*). É o análogo, no mundo de arquivo, da separação metadados/dados do pNFS e do NASD — e é o elo que liga NAS, SAN e nível terciário numa única arquitetura. **Declaração de escopo:** não aprofundamos o NDMP porque o enunciado não o lista; registramos por ser a resposta correta à pergunta “como se faz *backup* de um NAS?”.

Estabelecido o papel de cada arquitetura, resta a escolha de mídia. Silberschatz define o nível terciário como o de “fita magnética e *jukeboxes* de disco óptico” [2]. Elmasri e Navathe: “discos ópticos e fitas são mídias removíveis usadas nos sistemas de hoje como armazenamento *offline* para arquivamento de bancos de dados” [1]. As duas opções realistas em 2026 são **fita** e **armazenamento por objetos em classe de arquivamento**.

Fita: viva, e ligada à SAN

O LTO Program publica para a geração 10, lançada em 2025: cartuchos de “**30 TB e 40 TB de capacidade nativa**”, atingindo “75 TB e 100 TB respectivamente com razão de compressão de 2,5:1”, e “velocidades máximas de transferência de **400 MB/s (nativa)** e 1.200 MB/s (comprimida a 2,5:1, usando a interface *Fibre Channel* de 32 Gb)” [21]. **Proveniência:** especificação do consórcio LTO.

Atualizando o Elmasri & Navathe sobre fita

O livro registra o estado da tecnologia de fita em 2016: “o consórcio LTO [...] lançou o padrão LTO-6 mais recente em 2012 [...] A geração atual de bibliotecas usa *drives* LTO-6, com cartucho de 2,5 TB e taxa de transferência de 160 MB/s” [1]. Confrontando com a especificação vigente [21]:

$$\text{capacidade: } \frac{40 \text{ TB}}{2,5 \text{ TB}} = 16\times \quad \text{taxa: } \frac{400 \text{ MB/s}}{160 \text{ MB/s}} = 2,5\times$$

Consequência prática, e ela é contraintuitiva: em dez anos a capacidade cresceu 16× e a taxa apenas 2,5×. **O tempo para ler um cartucho cheio, portanto, aumentou $\approx 6,4\times$** — de $2,5 \times 10^{12} \div 160 \times 10^6 = 4,3 \text{ h}$ no LTO-6 para 27,8 h no LTO-10 (Seção 8.3). A fita ficou muito mais densa e proporcionalmente muito mais lenta de esvaziar, o que desloca a decisão de arquivamento do custo por TB para o **tempo de recuperação**. **Categoria:** atualização de dado datado, com derivação nossa da consequência.

Rótulo obrigatório: nativo ≠ comprimido

A indústria de fita publica *sempre* os dois números, e a diferença é de 2,5× — assumida, não medida. **Dados de banco de dados já comprimidos pelo próprio SGBD não atingem 2,5:1**, e frequentemente não passam de 1,2:1. Dimensionar uma biblioteca de fitas pela capacidade comprimida de catálogo é o erro mais comum do planejamento de *backup*. Neste relatório, sempre que citamos capacidade de fita, o número nativo é o número.

Achado da nossa verificação por script: a especificação do LTO não fecha

A regra do grupo de *refazer toda conta citada* pegou uma inconsistência na fonte primária. O LTO Program publica, para a mesma geração 10 [21]:

- capacidade: 30 e 40 TB nativos, “75 TB e 100 TB respectivamente com razão de compressão de 2,5:1” — **fecha**: $30 \times 2,5 = 75$ e $40 \times 2,5 = 100$;
- taxa: “400 MB/s (nativa) e 1.200 MB/s (comprimida a 2,5:1)” — **não fecha**: $400 \times 2,5 = 1.000$, e não 1.200. Os 1.200 implicariam razão de 3:1.

Não sabemos qual dos dois números a fonte pretendia, e não o adivinhamos. Registramos a inconsistência e adotamos, em todo este relatório, **apenas a taxa nativa de 400 MB/s** — que é a única não derivada de uma razão de compressão assumida, e a única que se aplica a dados de banco já comprimidos pelo SGBD. **Categoria: divergência na fonte primária, declarada.**

Note-se ainda o detalhe revelador na especificação: a taxa comprimida é declarada “*usando a interface Fibre Channel de 32 Gb*” [21]. Ou seja: a biblioteca de fitas moderna é um **dispositivo da SAN**. O nível terciário não é um mundo à parte — ele é conectado pela mesma *fabric* do nível secundário, e o dimensionamento do enlace FC precisa contemplar a janela de *backup*.

Objeto em classe de arquivamento

A alternativa é o armazenamento por objetos em classe fria. A AWS publica, para o S3 Glacier Deep Archive, preço de “**US\$ 0,00099 por GB-mês (ou US\$ 1 por TB-mês)**” e prazos de recuperação “**de 12 a 48 horas**” [23]. **Proveniência**: preço de tabela do fabricante, **consultado em setembro de 2026**; preço de nuvem é número volátil e não deve ser reproduzido sem data.

Os prazos de recuperação são justamente o que faz dessa classe um armazenamento *terciário* na definição de Silberschatz: ela não é acessível sem uma operação de restauração explícita, do mesmo modo que uma fita precisa ser montada. Para comparação, o mesmo documento da AWS registra recuperação em “milissegundos” para o *Glacier Instant Retrieval* e “tipicamente 3 a 5 horas” para a recuperação padrão do *Glacier Flexible Retrieval* [23].

Tabela 14: Recomendação para o nível terciário.

Critério	Fita (LTO em biblioteca)	Objeto em classe de arquivamento
Custo por TB armazenado	Menor no longo prazo; capital inicial alto (a biblioteca)	US\$ 1 por TB-mês no Deep Archive (set./2026) [23]; sem capital inicial
Custo de recuperação	Baixo e previsível (tempo de operador e de <i>drive</i>)	Cobrado por volume recuperado — é o custo que quebra orçamentos de restauração em massa
Prazo de recuperação	Minutos a horas (montagem + posicionamento sequencial)	12 a 48 h no Deep Archive [23]
Isolamento contra <i>ransomware</i>	Vantagem decisiva : o cartucho fora do <i>drive</i> é um <i>air gap</i> físico, inatingível por rede	Depende de <i>object lock</i> / WORM configurado corretamente; a proteção é lógica, não física
Longevidade e migração	Compatibilidade de leitura limitada a poucas gerações; exige migração periódica	Migração é responsabilidade do provedor
Recomendação	Arquivo legal de longo prazo, cópia final da regra 3-2-1, volumes muito grandes	Retenção operacional, conformidade com recuperação rara, quando não se quer operar <i>hardware</i>

A recomendação prática para o nível terciário

Não escolha: combine. A prática consolidada é a regra **3-2-1** — três cópias, em dois tipos de mídia, uma delas fora do sítio. Um desenho defensável para um SBD corporativo: (i) *backup* operacional em disco no NAS, para restauração rápida dos últimos dias; (ii) cópia em objeto, classe de arquivamento, para retenção de meses a anos; (iii) cópia em fita com cartucho ejetado, para o arquivo de longo prazo e como única cópia genuinamente inalcançável por um atacante com credenciais válidas. O item (iii) é o que a nuvem, por construção, não oferece.

9 Exemplos reais

Critério de seleção

O enunciado pede “exemplos reais, tais como aquele mostrado em sala de aula usando um vídeo do YouTube”. Não tivemos acesso ao vídeo específico exibido em aula. Em vez de supor qual era, adotamos um critério explícito: **só entram exemplos com documentação primária verificável** — artigo revisado por pares, documentação técnica do próprio operador, ou especificação de fabricante. Casos citados apenas em material de divulgação foram descartados.

(1) CERN — escala de exabyte com objeto, disco e fita

O CERN publica que o seu *data centre* “costumava **processar**, em média, um petabyte (um milhão de *gigabytes*) de dados por dia durante o LHC Run 2”, e que o plano para o Run 3 era armazenar “mais de 600 petabytes de dados”, com as instâncias do sistema EOS excedendo “sete bilhões de arquivos (em junho de 2022)” [25]. **Proveniência:** documentação do próprio operador; **data carimbada:** junho de 2022.

Rótulo: “processar” não é “gravar”

A nossa primeira versão escreveu que o CERN “gravou 1 PB por dia”. O texto do CERN diz *process* [25], e processar inclui o que é filtrado e descartado pelos sistemas de *trigger* — que descartam a esmagadora maioria dos eventos. A correção foi apontada pela rodada de verificação adversarial e é exatamente o tipo de erro que este relatório classifica como “número certo descrevendo outra coisa”.

A arquitetura combina os três paradigmas: EOS como camada de disco distribuída, CTA (*CERN Tape Archive*) como camada de fita, e Ceph para armazenamento de bloco e objeto da infraestrutura de nuvem interna [27]. É a demonstração mais clara de que, em escala extrema, a pergunta deixa de ser “NAS ou SAN” e passa a ser “qual camada para qual etapa do ciclo de vida do dado”.

(2) Dropbox — a migração de volta para infraestrutura própria

O Dropbox operou por anos sobre o Amazon S3 e migrou aproximadamente **500 petabytes** de dados de usuário para o seu próprio sistema de armazenamento por objetos, o **Magic Pocket**. **Precisão de rótulo:** a publicação do próprio Dropbox, de julho de 2016, declara que “agora servimos **90%** dos dados de nossos clientes no Magic Pocket” [26] — 90%, não 100%. A identificação do S3 como origem vem da cobertura de imprensa da época, não do texto do Dropbox; registramos a distinção. É o contraexemplo instrutivo: em escala suficientemente grande e com carga suficientemente previsível, a economia de escala do provedor deixa de compensar a margem que ele cobra. **Ressalva de rótulo:** o caso do Dropbox não é generalizável — exige volume de exabyte, equipe de engenharia de armazenamento dedicada e um padrão de acesso homogêneo. É o oposto do perfil de uma instalação corporativa típica de SGBD.

(3) Amazon Aurora — redesenhar a fronteira em vez de escolher o lado

Discutido na Seção 8.2. Os elementos verificáveis: replicação em “6 votos ($V = 6$), quórum de escrita de 4/6 ($V_w = 4$) e quórum de leitura de 3/6 ($V_r = 3$)” sobre três zonas de disponibilidade; volumes particionados em “segmentos de tamanho fixo pequeno, atualmente de 10 GB”; e a justificativa de que “um segmento de 10 GB pode ser reparado em 10 segundos num enlace de rede de 10 Gb/s” [24]. **Proveniência:** artigo revisado por pares (SIGMOD 2017).

(4) Oracle Direct NFS — NAS levado a sério para banco de dados

Discutido na Seção 3.4. É o exemplo real mais próximo do cotidiano de um DBA: a Oracle suporta oficialmente bancos de produção sobre NFS, desde que através do seu próprio cliente, com suporte documentado a “NFSv3, NFSv4, NFSv4.1 e pNFS” [17].

(5) SQL Server sobre SMB 3 — SAN dispensada em ambiente Windows

Discutido na Seção 3.3. A Microsoft documenta o suporte desde o SQL Server 2012, com o requisito explícito de *transparent failover* para cargas críticas [18]. É o caminho pelo qual muitas instalações Windows substituíram a SAN FC por um *Scale-Out File Server* sobre Ethernet.

10 Correções factuais ao material-fonte

Seguindo a orientação de verificar as afirmações das próprias fontes, registramos as divergências encontradas. O tom é de contribuição, não de reparo: são livros excelentes, e os pontos abaixo refletem sobretudo o envelhecimento natural de edições de 2016 e 2018 num campo que se move rápido.

Tabela 15: Divergências encontradas no material-fonte, com correção e consequência prática.

Fonte	Afirmação	Correção	Consequência prática
Silberschatz §12.2	“SATA-3 nominalmente suporta 6 gigabytes por segundo, permitindo velocidades de transferência de até 600 megabytes por segundo” [2]	São 6 gigabits por segundo. O próprio “600 MB/s” na mesma frase prova: $6 \text{ Gb/s} \div 8 \times (8/10, \text{ codificação } 8\text{b}/10\text{b}) = 600 \text{ MB/s}$. Se fossem 6 GB/s, seriam 6.000 MB/s.	Erro de bit/byte por fator de 8. É o exemplo canônico de por que este relatório verifica unidade antes de valor.
Elmasri §16.11.3	“FCoE [...] pode ser pensado como iSCSI sem o IP” [1]	FCoE encapsula <i>quadros FC completos</i> em Ethernet, exige DCB e não é roteável em L3 [7]	Um projetista que aceite a analogia tentará usar FCoE entre <i>data centers</i> , o que é impossível. Ver Seção 4.7.
Elmasri §16.2.1	“SATA agora é chamado de NL-SAS, de <i>nearline</i> SAS” [1]	Falso. NL-SAS = mecânica e mídia SATA <i>nearline</i> de 7.200 rpm com interface e protocolo SAS (porta dupla, conjunto completo de comandos SCSI). São coisas distintas.	Um <i>array</i> SAS com discos SATA exige STP/interposer e perde <i>multipath</i> . Confundir os dois leva a especificar <i>hardware</i> que não entrega a redundância esperada.
Elmasri §16.11.5	Seagate <i>Kinetic Open Storage</i> como o produto que viabilizou OSD [1]	Datado: a plataforma não vingou; o padrão de fato do armazenamento por objetos tornou-se a API HTTP do S3 .	Ver Seção 7. Não é erro, é envelhecimento.
LTO Program, página do LTO-10	“400 MB/s (nativa) e 1.200 MB/s (comprimida a 2,5:1)” [21]	Não fecha: $400 \times 2,5 = 1.000$. Os 1.200 implicariam 3:1. As capacidades, essas, fecham a 2,5:1	Adotamos só a taxa nativa. Achado da nossa verificação por script (Seção 13.5).
FCIA <i>roadmap</i> × convenção T11 × FC-PI-8	Três números diferentes são publicados como “128GFC”: 25.600 (ISL), 24.850 (serial Gen 8) e 12.800 (FC-PI-8 rev. 1.4) [9, 14]	Todos corretos, descrevendo coisas diferentes. A leitura <i>full-duplex</i> é derivação nossa, provada por física (a vazão publicada excede a taxa de linha).	Citar “128GFC” sem dizer qual dos três erra por até 2×. Ver Seção 4.2.

Sobre a ressalva do NL-SAS

Registramos, por rigor, que “NL-SAS” é **convenção de indústria, não termo normativo do T10**. A correção acima é sobre a natureza do dispositivo (mecânica SATA + interface SAS), não sobre uma definição formal — que não existe no padrão.

11 Conclusão

A comparação entre NAS e SAN é frequentemente apresentada como uma disputa de desempenho, e é provavelmente aí que ela é pior compreendida. A diferença real é de **unidade de abstração**: a SAN entrega blocos e deixa o servidor dono do sistema de arquivos; o NAS entrega arquivos e assume essa propriedade para si. Todas as demais diferenças — protocolo, cabeamento, custo, modelo de falha, forma de compartilhamento — decorrem dessa escolha, e não o contrário.

Três conclusões se destacam.

Primeira: o argumento de velocidade envelheceu. A latência de um salto em *switch Fibre Channel* de geração corrente é de 460 ns [15], contra 20 a 100 μ s de uma leitura aleatória em SSD [2] — menos de 5% do tempo total do acesso (derivação da Seção 4.3). A rede saiu do caminho crítico. O que sobra como diferença entre as arquiteturas é semântico: quem garante a atomicidade da página, quem arbitra o travamento, o que acontece quando o caminho oscila. É por isso que a Microsoft, ao suportar SQL Server sobre SMB, não fala de banda, e sim de *transparent failover* [18]; e é por isso que a Oracle reimplementou o cliente NFS dentro do próprio motor [17].

Segunda: o rótulo é tão importante quanto o valor. Este trabalho encontrou, num campo inteiramente normatizado, cinco casos em que números corretos descrevem coisas diferentes: o valor “128GFC” é publicado em **três escalas distintas** — 25.600 (ISL), 24.850 (serial Gen 8) e 12.800 (o herdado do FC-PI-8) [9, 14]; o LTO publica capacidade nativa e comprimida a 2,5:1, diferença de $2,5\times$ [21]; o Silberschatz escreveu *gigabytes* onde eram *gigabits*, diferença de $8\times$ [2]; a depreciação do AFP separa cliente de servidor por cinco anos [20]; e os números do experimento do Aurora são totais de uma janela de 30 minutos numa carga sintética só de escrita, não vazões absolutas [24]. Nenhum é erro de pesquisa — todos são erros de *leitura*, e passariam por uma revisão que só conferisse se o número aparece na fonte.

E a lição mais dura veio de um erro nosso. A primeira versão deste relatório afirmava que a diferença entre as duas escalas de FC era “de exatos $2\times$ ”. É verdade de 1GFC a 64GFC e **falso na geração vigente**: o 128GFC serial entrega 24.850 MB/s, e $24.850 \div 2 = 12.425 \neq 12.800$. A regra que enunciamos como âncora quebrou exatamente na geração que estávamos descrevendo, e só descobrimos isso porque a segunda rodada adversarial (Seção 13.6) foi buscar a tabela que a primeira versão declarara inacessível. **Um trabalho que se propõe a auditar rótulos não pode falhar em rótulos** — e o registro dessa falha é a parte deste relatório que mais nos ensinou.

Terceira: a pergunta do enunciado está sendo reformulada pela indústria. O AST tenta resolver, no *array* e com granularidade de 256 MiB e latência de horas, um problema que o *buffer manager* do SGBD já resolve com granularidade de 8 a 16 KiB e latência de acesso — uma diferença de 16.384 a 32.768 vezes, conforme o SGBD (Seção 6.3). O armazenamento por objetos abandonou tanto o bloco quanto o arquivo e venceu completamente nos domínios em que a latência não importa. E o Aurora mostrou que, quando o motor deixa de escrever páginas e passa a escrever apenas *log*, a escolha entre bloco e arquivo perde parte de seu sentido [24]. Continua havendo uma resposta certa para “NAS ou SAN?” em cada carga concreta — e a Seção 8 a dá — mas vale reconhecer que a fronteira arquitetural se moveu.

12 Questões em aberto para o debate

Questões que consideramos genuinamente abertas, e que oferecemos para a discussão em sala e no fórum:

1. **Se a rede FC responde por menos de 5% da latência de um acesso NVMe, o que ainda justifica economicamente uma *fabric* FC dedicada?** A resposta é isolamento operacional e previsibilidade, ou é inércia de investimento — exatamente a inércia que Elmasri e Navathe apontam ao explicar a adoção lenta do iSCSI em grandes empresas [1]?
2. **O AST pode ser *contraproducente* para OLTP?** Se o *buffer pool* absorve os acessos ao dado mais quente, o AST enxerga esse dado como frio e pode rebaixá-lo. Existe alguma implementação que exponha as estatísticas do *buffer manager* ao *array*? Deveria existir — ou a camada de armazenamento deve permanecer deliberadamente ignorante?
3. **Qual é a granularidade certa para *tiering*?** 256 MB [16] é $16.384\times$ maior que uma página de 16 KiB. Existe um ponto ótimo, ou o problema é que *tiering* e *caching* são mecanismos diferentes que a indústria vende com o mesmo nome?
4. **Com o S3 fornecendo consistência forte desde 2020 [22], qual é a próxima barreira real ao armazenamento por objetos como camada primária de um SGBD?** A latência, a imutabilidade do objeto, ou a ausência de ordenação de escrita entre objetos?

5. **A convergência (FCoE) fracassou, ou apenas mudou de nome?** O FCoE prometia uma rede única e ficou num nicho. O NVMe/TCP promete o mesmo com outra pilha. O que mudou tecnicamente para que a segunda tentativa tenha chance — ou é a mesma aposta com uma sigla nova?
6. **Fita ainda tem futuro, ou o *air gap* é o último argumento?** O LTO-10 dobrou a capacidade sem aumentar a taxa: 40 TB por cartucho a 400 MB/s nativos [21] significa mais de 27 horas para ler um cartucho cheio (derivação: $40 \times 10^{12} \div (400 \times 10^6) = 100.000 \text{ s} = 27,8 \text{ h}$). A capacidade cresce e a velocidade não. Em que ponto isso torna a recuperação inviável na prática?
7. **Se o Aurora mostrou que “o log é o banco de dados” [24], por que os SGBDs relacionais tradicionais não seguiram o mesmo caminho?** É limitação técnica, ou é que essa arquitetura só faz sentido quando o mesmo fornecedor controla o motor e o armazenamento?

Referências

[Livros] Livros-texto

- [1] ELMASRI, R.; NAVATHE, S. B. *Sistemas de Banco de Dados*. 7. ed. São Paulo: Pearson, 2018. Capítulo 16 — *Disk Storage, Basic File Structures, Hashing, and Modern Storage Architectures*, em especial a Seção 16.11 (*Modern Storage Architectures*).
- [2] SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistemas de Banco de Dados*. 7. ed. Rio de Janeiro: Gen/LTC, 2020. Capítulo 12 — *Physical Storage Systems*, em especial a Seção 12.2 (*Storage Interfaces*).
- [3] GARCIA-MOLINA, H.; ULLMAN, J. D.; WIDOM, J. *Database Systems: The Complete Book*. 2. ed. Upper Saddle River: Pearson Prentice Hall, 2008. Capítulo 13 — *Secondary Storage Management*.

[Normas] Normas e especificações

- [4] IETF. *RFC 8881 — Network File System (NFS) Version 4 Minor Version 1 Protocol*. 2020. Obsoleta a RFC 5661. Disponível em: <https://www.rfc-editor.org/info/rfc8881/>.
- [5] IETF. *RFC 7143 — Internet Small Computer System Interface (iSCSI) Protocol (Consolidated)*. Abr. 2014. Obsoleta a RFC 3720. Disponível em: <https://www.rfc-editor.org/info/rfc7143/>.
- [6] IETF. *RFC 3821 — Fibre Channel Over TCP/IP (FCIP)*. Jul. 2004. Disponível em: <https://www.rfc-editor.org/info/rfc3821/>.
- [7] INCITS/T11. *FC-BB-5 — Fibre Channel Backbone-5*, publicado como **ANSI/INCITS 462-2010**. Define o encapsulamento FCoE em quadro Ethernet sob o *EtherType* 0x8906 e o protocolo FIP. Sucedido pelo FC-BB-6 (VN2VN). Registro do comitê disponível em: <https://www.t11.org/>.
- [8] IEEE. *IEEE Std 802.1Qbb — Priority-based Flow Control*. Mecanismo de pausa por prioridade, *enlace a enlace*, que constitui a Ethernet sem perdas exigida pelo FCoE. Incorporado ao IEEE Std 802.1Q. Disponível em: https://standards.ieee.org/standard/802_1Qbb-2011.html.
- [9] FIBRE CHANNEL INDUSTRY ASSOCIATION. *The Fibre Channel Roadmap — speedmap* vigente (v24, jul./2023), com tabelas separadas para FC serial e para *Inter-Switch Link*. Consultado em set./2026. Disponível em: <https://fibrenchannel.org/roadmap/>.
- [10] THE REGISTER. *Seagate’s Kinetic drives: HGST drew a blank looking for use cases*. 17 mar. 2016. Disponível em: https://www.theregister.com/2016/03/17/seagate_has_seen_the_light/.
- [11] MICROSOFT. *Detect, enable, and disable SMBv1, SMBv2, and SMBv3 in Windows*. Microsoft Learn. Disponível em: <https://learn.microsoft.com/en-us/windows-server/storage/file-server/troubleshoot/detect-enable-and-disable-smbv1-v2-v3>.
- [12] *nfs(5) — Linux manual page*. Seção *Data and Metadata Coherence*. Disponível em: <https://man7.org/linux/man-pages/man5/nfs.5.html>.

[Fabricantes] Fabricantes e consórcios

- [13] FIBRE CHANNEL INDUSTRY ASSOCIATION. *FCIA Official Speedmap*, v21, 1º dez. 2016. *Versão superada*, citada aqui apenas no registro do erro da Seção 4.2. Disponível em: https://fibrenchannel.org/wp-content/uploads/2015/10/FCIA_SPEEDMAP_v21.pdf.
- [14] FIBRE CHANNEL INDUSTRY ASSOCIATION. *128GFC Q&A*. Disponível em: <https://fibrenchannel.org/128gfc-qa/>.
- [15] BROADCOM. *Brocade G710 Switch — Product Brief*. Disponível em: <https://docs.broadcom.com/doc/G710-Switch-PB>.
- [16] DELL TECHNOLOGIES. *Dell EMC Unity: FAST Technology Overview*. Technical White Paper H15086.3. Disponível em: <https://www.delltechnologies.com/asset/en-us/products/storage/industry-market/h15086-emc-unity-fast-technology-overview.pdf>.
- [17] ORACLE. *About Direct NFS Client Mounts to NFS Storage Devices*. Oracle Database Installation Guide 19c. Disponível em: <https://docs.oracle.com/en/database/oracle/oracle-database/19/ssdbi/about-direct-nfs-client-mounts-to-nfs-storage-devices.html>.
- [18] MICROSOFT. *Install SQL Server with SMB Fileshare Storage*. Microsoft Learn. Disponível em: <https://learn.microsoft.com/en-us/sql/database-engine/install-windows/install-sql-server-with-smb-fileshare-as-a-storage-option>.
- [19] ORACLE. *MySQL 8.4 Reference Manual, §17.6.4 Doublewrite Buffer*. Disponível em: <https://dev.mysql.com/doc/refman/8.4/en/innodb-doublewrite-buffer.html>.
- [20] APPLE. *What’s new for enterprise in macOS Sequoia — notas corporativas do macOS Sequoia 15.5 (2025)*. Texto literal: “*Apple Filing Protocol (AFP) client is deprecated and will be removed in a future version of macOS*”. Documento de suporte 121011. Disponível em: <https://support.apple.com/121011>.
- [21] LTO PROGRAM. *LTO-10: LTO Generation 10 Technology*. Disponível em: <https://www.lto.org/lto-10/>.
- [22] AMAZON WEB SERVICES. *Amazon S3 FAQs*. Disponível em: <https://aws.amazon.com/s3/faqs/>.
- [23] AMAZON WEB SERVICES. *Amazon S3 Glacier storage classes*. Preços e prazos consultados em set. 2026. Disponível em: <https://aws.amazon.com/s3/storage-classes/glacier/>.

[Artigos] Artigos e documentação de operadores

- [24] VERBITSKI, A. *et al. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases*. In: Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD ’17). Disponível em: <https://pages.cs.wisc.edu/~xyx/cs764-f20/papers/aurora-sigmod-17.pdf>.
- [25] CERN. *Storage*. Página institucional de computação. Dados de junho de 2022. Disponível em: <https://home.cern/science/computing/storage>.
- [26] DROPBOX. *Moving 500 petabytes of user data into our Magic Pocket*. Disponível em: <https://blog.dropbox.com/topics/technology/moving-500-petabytes-of-user-data-into-our-magic-pocket>.
- [27] CERN. *CERN Tape Archive: Run 3 Production Experience — CTA Tier-0 service performance during the start of the LHC Run 3 and the various lessons learnt*. EPJ Web of Conferences (CHEP 2024). Disponível em: https://www.epj-conferences.org/articles/epjconf/abs/2024/05/epjconf_chep2024_01049/epjconf_chep2024_01049.html.
- [28] MACRUMORS. *macOS 27 Golden Gate Kills Time Capsule Support*. 17 jun. 2026. Cobertura da ausência do cliente AFP nos betas do macOS 27. Disponível em: <https://www.macrumors.com/2026/06/17/macOS-27-golden-gate-kills-time-capsule-support/>.

13 Post-Mortem

O enunciado exige esta seção e especifica seu conteúdo: (i) o log dos *prompts*-chave, (ii) a demonstração de autoria — quem fez o quê, quem decidiu o quê e por quê — e (iii) quem corrigiu e otimizou o que a IA fez de errado, listando os erros gerados e os acertos feitos. Tratamos a seção como parte substantiva do trabalho, e não como formalidade: é aqui que o processo fica auditável.

13.1 Como a IA foi usada, e como não foi

A IA generativa (Claude, da Anthropic) foi usada como **pesquisadora e redatora sob supervisão**, nunca como fonte. A regra que o grupo fixou no início, e que estruturou todo o resto, foi:

Regra de trabalho adotada pelo grupo

A IA não é fonte de fato. Todo número, data e citação deste relatório foi buscado em documento primário — RFC, norma T11, *white paper* de fabricante, artigo revisado por pares — e a URL correspondente está nas referências. Quando a IA produziu uma afirmação sem que encontrássemos fonte, a afirmação foi *removida* ou marcada como *não encontrada*. Declarar a lacuna é resultado; preencher com plausibilidade é fabricação.

A regra mudou o papel de todos: em vez de gastar tempo redigindo, o grupo gastou tempo *conferindo*. A Seção 13.4 mostra que foi a decisão certa — dezoito afirmações produzidas pela IA estavam erradas ou mal rotuladas, e **nenhuma delas era detectável por leitura atenta**, apenas por confronto com a fonte.

13.2 Log dos *prompts*-chave

O enunciado pede o *log* dos *prompts*-chave. Reproduzimos **na íntegra** os oito que determinaram o conteúdo, na ordem em que foram emitidos, com o efeito de cada um. Ficam de fora apenas os de execução mecânica (compilar, ajustar tabela, gerar figura), que não alteram conteúdo.

P1 — enquadramento • *fixou o enunciado como checklist literal*

“Me ajude a fazer esse trabalho, seguindo as instruções do projeto, e lendo o material do projeto (dois livros).” — acompanhado do texto integral do enunciado da tarefa.

P2 — decisão humana, antes de qualquer redação • *quatro perguntas de escopo*

“(a) Qual escopo para o relatório: mesmo porte da tarefa anterior (≈ 45 pág.), enxuto e denso ($\approx 25\text{--}30$) ou máximo possível? (b) O vídeo do YouTube mostrado em sala — você sabe qual é? (c) Como tratar a autoria no Post-Mortem? (d) Algum reforço específico além do enunciado?”

Respostas que mudaram o produto: enxuto e denso; o professor não passou o vídeo; propor uma divisão de autoria para o grupo ajustar; e mover os aprofundamentos para documento à parte.

P3 — ancoragem nas fontes • *impediu reconstrução de memória*

“Leia o Capítulo 12 do Silberschatz e o Capítulo 16 do Elmasri & Navathe anexados ao projeto e extraia o texto original exato sobre SAN, NAS, iSCSI, FCIP, FCoE, AST e object storage — não reconstrua de memória.”

P4 — pesquisa dirigida, um por item do enunciado • *cada resposta virou uma referência*

Emitidos em série, todos exigindo a norma de origem: “qual RFC define o iSCSI hoje e o que ela obsoleta?”; “qual norma T11 padroniza o FCoE e o que ela exige da rede Ethernet?”; “o AFP foi removido ou depreciado, e em qual versão — cliente ou servidor?”; “qual é a granularidade de relocação do FAST VP, com fonte?”; “qual é a capacidade e a taxa nativas do LTO-10, e qual razão de compressão o consórcio assume?”; “o S3 ainda é eventualmente consistente?”.

P5 — conferir a própria fonte • *produziu a Seção 10*

“Verifique o que os próprios livros afirmam sobre SATA-3, NL-SAS, FCoE e Fibre Channel, e confronte com as normas. Se algum estiver errado, documente com fonte e consequência prática.”

P6 — primeira rodada adversarial • *48 afirmações, 16 correções*

“Você é um verificador de fatos adversarial. Verifique cada afirmação abaixo com busca web e classifique como CONFIRMADO (com URL), IMPRECISO (com o valor correto), NÃO VERIFICÁVEL ou FALSO. Seja implacável: o objetivo é encontrar erros antes que o professor os encontre. **Não confirme nada por plausibilidade — só com fonte.** Se não achar fonte, diga NÃO VERIFICÁVEL, o que já é um achado. Ao final, ordene por gravidade o que um avaliador rigoroso poderia usar para derrubar o trabalho.”

P7 — verificação por script • expôs a inconsistência do LTO

“Escreva um script que (1) refaça todas as contas publicadas no relatório e compare com o valor impresso; (2) cruze os números-chave entre o `.tex` e o `.pptx`; (3) confira a checklist do enunciado contra o `.tex`. Ele deve falhar com código de erro se qualquer item não fechar.”

P8 — segunda rodada: simulação da correção • 15 correções, 3 gravíssimas

“Você é a IA que o professor usa para **corrigir** trabalhos. Seu trabalho não é elogiar. Leia o relatório e os slides na íntegra. (1) Extraia as afirmações mais arriscadas e verifique com busca web — inclusive **as correções que o grupo alega ter encontrado nos livros: elas estão certas mesmo?** (2) Produza no mínimo 20 perguntas que encurralem o apresentador, indicando por que cada uma é perigosa, qual seria a resposta correta e o nível de risco. (3) Liste todos os defeitos ordenados por gravidade, incluindo os dos slides como material de apresentação. (4) Dê uma nota de 0 a 10 decomposta por critério, e diga o que separaria este trabalho de um 10.”

O que o log mostra sobre a estrutura dos prompts

Os oito se dividem em três funções, e a proporção importa: **um** prompt de produção (P1), **três** de coleta com proveniência (P3–P5) e **três** de destruição do próprio resultado (P6–P8), mais um de decisão humana (P2). **A maior parte do esforço de prompting foi gasta tentando derrubar o que a IA havia escrito**, não pedindo que escrevesse mais. Os prompts P6 e P8 produziram, juntos, 31 das 33 correções da Seção 13.4.

13.3 Autoria: quem fez o quê, quem decidiu o quê e por quê

Tabela 16: Divisão de trabalho e de decisão.

Integrante	Fez	Decidiu, e por quê
Bernardo Brandão Pozzato Carvalho Costa	Seção 4 (SAN, pilha FC, endereçamento, zoneamento) e Seção 4.3 (topologias).	Decidiu tratar “FC <i>Switch</i> ” como topologia , não como protocolo, apesar de o enunciado listá-lo entre as “variações”. Razão: é assim que a norma T11 o define, e apresentá-lo como protocolo produziria uma tabela conceitualmente errada. Optou por atender o item do enunciado e explicar a distinção.
Enzo de Carvalho Sampaio	Seções 4.5, 4.6 e 4.7 (iSCSI, FCIP, FCoE); derivação da latência de propagação.	Decidiu incluir a derivação física da distância (1 ms por 100 km) em vez de repetir a frase de catálogo “FCIP serve para longas distâncias”. Razão: o número explica <i>por que</i> a replicação intercontinental precisa ser assíncrona, e é defensável em arguição. Decidiu também registrar NVMe-oF apenas como nota de fronteira, para não extrapolar o escopo do enunciado.
Gabriel Schmitz Corrêa Ritzawinsk	Seção 3 (NAS) e os três protocolos: SMB/CIFS, NFS e AFP.	Decidiu não fabricar uma aplicação de banco de dados para o AFP. Razão: não existe SGBD de porte suportado sobre AFP, e inventar um caso de uso seria exatamente o preenchimento que o grupo proibiu. Transformou o item numa análise do fim de vida do protocolo, que é o fato relevante. Foi quem localizou a distinção cliente × servidor na depreciação.
Guilherme En Shih Hu	Coordenação, integração das seções, condução das <i>duas</i> rodadas adversariais, redação deste Post-Mortem.	Decidiu escopo enxuto (≈25–30 páginas) em vez de repetir o porte da tarefa anterior. Razão: cobrir a lista do enunciado com profundidade e sobrar tempo real de revisão vale mais que volume — e foi a revisão que encontrou os erros. Decidiu mover os aprofundamentos para documento separado. Após a segunda rodada, decidiu registrar os três erros gravíssimos no corpo do texto (Seções 4.2 e 4.7) em vez de corrigi-los em silêncio — razão: um trabalho que se apresenta como auditoria de rótulos ganha mais credibilidade mostrando onde a própria auditoria falhou do que exibindo um texto sem cicatrizes.
Raphael Henrique da Silva Pereira	Seções 6 (AST) e 7 (<i>object storage</i>).	Decidiu adotar a tensão entre AST e <i>buffer manager</i> como eixo analítico da seção de <i>tiering</i> (Seção 6.3). Razão: nenhum dos três livros faz essa ligação, e ela converte um item meramente descritivo (“comente sobre AST”) em contribuição própria. Decidiu também re-verificar se a objeção clássica de consistência eventual do S3 ainda valia — não valia, e a verificação corrigiu um erro da IA.
Vivian Maria da Silva e Souza	Seções 8 (recomendação; níveis secundário e terciário) e 9 (exemplos reais).	Decidiu o critério de seleção dos exemplos : só entram casos com documentação primária verificável. Razão: sem saber qual era o vídeo citado em aula, a alternativa honesta a adivinhar era declarar um critério e aplicá-lo. Decidiu estruturar a recomendação por perfil de carga , e não por porte de empresa, porque é o perfil de carga — e não o tamanho da organização — que determina se o motor precisa de bloco ou de arquivo.

As quatro decisões humanas que mais mudaram o resultado

- (1) **Escopo enxuto.** Trocar volume por revisão. Sem essa decisão não haveria tempo para a rodada adversarial, que foi o que encontrou os erros graves.
- (2) **Não adivinhar o vídeo.** O enunciado cita “aquele mostrado em sala de aula usando um vídeo do YouTube”. Não sabíamos qual era. A IA, deixada solta, teria produzido um exemplo plausível. O grupo decidiu declarar a lacuna e definir um critério de substituição (Seção 9).
- (3) **Conferir os próprios livros.** Decisão herdada da tarefa anterior; rendeu três correções ao material-fonte (Seção 10).
- (4) **Exigir proveniência de todo número.** Foi o que tornou a rodada adversarial possível: só se pode verificar o que está declarado.

13.4 Erros gerados pela IA e correções aplicadas

Lista completa das **33 correções** aplicadas em duas rodadas. A coluna “conferido na fonte por” registra quem foi ao documento primário confirmar a correção antes de ela entrar no texto — distinção que importa, porque *encontrar* um erro e *confirmar* a correção são atos diferentes, e o enunciado pergunta pelo segundo.

Primeira rodada (18 correções)

#	Erro gerado pela IA	Correção aplicada	Encontrado por	Conferido na fonte por
1	“A versão de remoção do cliente AFP ainda não foi anunciada.”	Verdadeiro em mai./2025, falso em set./2026: o macOS 27 já não traz o cliente AFP nos <i>betas</i> de jun./2026 [28].	Rodada 1	Gabriel Schmitz
2	“A FCIA publica <i>full-duplex</i> ”, apresentado como citação.	Nenhum documento da FCIA diz isso. Re-classificado como derivação nossa.	Rodada 1	Bernardo Brandão
3	<i>Speedmap</i> v21 (2016) apresentado como vigente.	Data explicitada; ressalva de revisão acrescentada. <i>Correção incompleta — ver #19.</i>	Rodada 1	Bernardo Brandão
4	“Codificação 64b/66b a partir de 16GFC.”	64b/66b só no 16GFC; de 32GFC em diante é 256b/257b com RS-FEC.	Rodada 1	Bernardo Brandão
5	“CIFS $\equiv$ SMB 1.0.”	A Microsoft diz que o CIFS é um <i>dialeto</i> do SMB.	Rodada 1	Gabriel Schmitz
6	“SMB 3.1.1 usa AES-GCM”, sem tamanho de chave.	AES-128-CCM no 3.0; AES-128-GCM no 3.1.1.	Rodada 1	Gabriel Schmitz
7	“SMBv1 fora por padrão a partir do Server 2019.”	O primeiro Server foi a versão 1709 (SAC); o 2019 é o primeiro LTSC.	Rodada 1	Gabriel Schmitz
8	“FC-AL suporta até 126 dispositivos.”	126 NL_Ports + 1 FL_Port = 127 endereços.	Rodada 1	Bernardo Brandão
9	Reproduziu “7,7 vezes menos E/S” do Aurora sem refazer a conta.	$7,4 \div 0,95 = 7,79 \rightarrow 7,8\times$. Divergência registrada.	Rodada 1	Vivian Maria
10	“O CERN gravou 1 PB por dia.”	O CERN escreve <i>process</i> .	Rodada 1	Vivian Maria
11	“O Dropbox concluiu a migração em 2016.”	O texto diz “90% dos dados de clientes” em jul./2016.	Rodada 1	Vivian Maria
12	“A plataforma Seagate Kinetic foi descontinuada.”	Sem fonte. Trocado por “não obteve tração comercial” [10].	Rodada 1	Raphael Henrique
13	“256 MB é 16.384×16 KiB”, sem declarar convenção.	Convenção binária explicitada. <i>Ampliado em #31.</i>	Rodada 1	Raphael Henrique
14	“ $14,025 \times 64/66 \approx 1.600$ MB/s.”	Dá 1.700; os 1.600 são convenção de nomenclatura, não resultado da conta.	Rodada 1	Bernardo Brandão
15	Citou a RFC 3720 como norma vigente do iSCSI.	Obsoletada pela RFC 7143 [5].	Grupo (pesquisa)	Enzo de Carvalho
16	Citou a RFC 5661 como norma do NFSv4.1.	Obsoletada pela RFC 8881 [4].	Grupo (pesquisa)	Gabriel Schmitz
17	“O Amazon S3 é eventualmente consistente.”	Consistência forte desde dez./2020 [22].	Grupo (pesquisa)	Raphael Henrique
18	Reproduziu dos livros “FCoE é iSCSI sem o IP” e “SATA agora é NL-SAS”.	Viraram a Seção 10. <i>A primeira foi refeita em #21.</i>	Grupo (leitura)	Enzo de Carvalho

Segunda rodada (15 correções) — a simulação de correção

#	Defeito da versão 1	Correção aplicada	Gravidade	Conferido na fonte por
19	Declaramos uma lacuna que não existia: “não conseguimos acessar a tabela numérica da v24”.	A v24 está em HTML na página de <i>roadmap</i> da FCIA [9]. Tabela refeita, lacuna falsa removida, erro registrado no corpo do texto.	GRAVÍSSIMA	Bernardo Brandão
20	“Diferença de exatos 2×” entre FCIA e T11, enunciado como conclusão.	Falso na Gen 8: $24.850 \div 2 = 12.425 \neq 12.800$. Reescrito, e a quebra da convenção virou o achado principal da seção.	GRAVÍSSIMA	Bernardo Brandão
21	Correção do FCoE sobre citação recortada: duas das “três razões” já estavam no parágrafo do livro.	Parágrafo completo citado; correção reduzida a duas imprecisões reais (“fim-a-fim” × enlace a enlace; não-roteabilidade em L3).	GRAVÍSSIMA	Enzo de Carvalho
22	Aurora apresentado sem a janela do experimento.	Acrescentado: 30 minutos, SysBench só de escrita, 100 GB, r3.8xlarge [24].	Grave	Vivian Maria
23	“<5% do tempo do acesso” com denominador incompleto.	Escopo declarado (só comutação); lacuna da latência de controladora declarada; ressalva de data acrescentada.	Grave	Bernardo Brandão
24	Afirmções comparativas de custo sem proveniência, violando a própria metodologia.	Reclassificadas como estruturais; lacuna declarada explicitamente.	Grave	Vivian Maria
25	Citação da Apple entre aspas sem ser literal.	Texto literal repostado [20].	Média	Gabriel Schmitz
26	Citação da RFC 8881 sem a condição normativa.	Acrescentada a condição SEQUENCE/CB_SEQUENCE.	Média	Gabriel Schmitz
27	“Podem ser desligados quando o dispositivo garante escrita atômica.”	O manual restringe a Fusion-io NVMFS em Linux [19].	Média	Raphael Henrique
28	“Atualização” sobre a API do S3 vendida como achado.	O livro já registra S3, Azure e o GET/PUT do Swift. Reconhecido no texto.	Média	Raphael Henrique
29	AFP tratado só como obituário; o enunciado pede “como funciona”.	Acrescentada a mecânica: DSI/548, sessão, <i>forks</i> , FPBYTERANGELOCK, codificação.	Grave	Gabriel Schmitz
30	Tabela do NFS induzia a ler v4.2 (2016) antes de v4.1 (2020).	Acrescentada coluna “introduzida em” e nota de leitura.	Média	Gabriel Schmitz
31	Fator do AST oscilava entre 8 e 16 KiB.	Publicados os dois ($16.384\times$ e $32.768\times$); adotado o menor por ser conservador.	Média	Raphael Henrique
32	Negativos universais (“nenhum livro”, “não existe SGBD”).	Reduzidos ao alcance efetivamente verificado.	Média	Guilherme En Shih
33	A aritmética do próprio Post-Mortem não fechava: 11 ressalvas + 3 IMPRECISO + 1 FALSO + 1 NÃO VERIFICÁVEL = 16, e a tabela declarava 14.	Recontado. A única conta que não refizemos era a nossa.	Grave	Guilherme En Shih

Padrão dos erros — a lição que levamos

Das 33 correções, **nenhuma** foi invenção pura do tipo que se costuma chamar de “alucinação”. Quatro famílias, agora que temos duas rodadas para comparar:

(a) **Rótulo errado sobre número certo** — #2, #4, #6, #7, #8, #10, #13, #14, #22, #23, #25, #26, #27, #31: **quatorze casos**, a família mais numerosa e a que sobreviveu à primeira rodada. O valor existe na fonte, mas descreve outra coisa. **Antídoto:** perguntar sempre “este número mede exatamente o quê, em que condições?”.

(b) **Desatualização** — #1, #3, #15, #16, #17, #19: seis casos. **Antídoto:** carimbar data e reverificar qual norma está vigente.

(c) **Excesso de confiança na formulação** — #5, #9, #11, #12, #24, #28, #32, #33: oito casos. “Não há fonte” vira afirmação categórica; conta citada não é refeita.

(d) **Recorte que favorece o próprio argumento** — #20, #21, #29: **três casos, e são os três mais graves**. Não são erros de fato: são erros de *enquadramento*. A IA citou o livro até a frase que sustentava a crítica e parou; enunciou como regra geral um padrão que valia só nas gerações que ela havia tabelado; tratou um item do enunciado pelo ângulo em que tinha material. **Nenhuma verificação de fatos pega isso** — só pega quem lê o parágrafo inteiro e pergunta “o que foi deixado de fora?”. É o achado metodológico mais importante deste trabalho.

13.5 Primeira rodada de verificação adversarial

Protocolo. Concluída a primeira versão completa, extraímos **48 afirmações verificáveis** (existência e especificação de produto, datas, números de mercado, taxas e citações atribuídas aos livros) e as submetemos a uma segunda passagem de IA, em sessão independente e sem acesso ao texto do relatório, com o seguinte *prompt*:

“Você é um verificador de fatos adversarial. Verifique cada afirmação abaixo com busca web e classifique como CONFIRMADO (com URL), IMPRECISO (com o valor correto), NÃO VERIFICÁVEL ou FALSO. Seja implacável: o objetivo é encontrar erros antes que o professor os encontre. **Não confirme nada por plausibilidade — só com fonte.** Se não achar fonte, diga NÃO VERIFICÁVEL, o que já é um achado. Ao final, ordene por gravidade o que um avaliador rigoroso poderia usar para derrubar o trabalho.”

Por que essas duas cláusulas. “Não confirme por plausibilidade” impede a validação complacente, que é o modo de falha natural de um verificador automático — sem ela, ele concorda com quase tudo. “Ordene por gravidade” força a perspectiva do avaliador, e é o que transforma uma lista de reparos numa lista de riscos.

Tabela 19: Resultado da rodada adversarial sobre 48 afirmações.

Veredito	Qtd.	Observação
CONFIRMADO	43	Dos quais 11 com ressalva de rótulo que exigiu ajuste de redação.
IMPRECISO	3	Codificação de linha do FC; a leitura <i>full-duplex</i> ; a conta do 16GFC.
FALSO	1	A afirmação sobre a remoção do cliente AFP (erro #1).
NÃO VERIFICÁVEL	1	A descontinuação da Seagate Kinetic (erro #12).
Total	48	16 correções aplicadas — as 5 não-confirmadas mais as 11 ressalvas de rótulo —, 3 delas graves.

Os três achados graves, na ordem de gravidade dada pelo verificador:

1. **AFP (erro #1).** Um relatório entregue em setembro de 2026 afirmando que a remoção “ainda não foi anunciada”, com o macOS 27 já em *beta* sem o cliente AFP, é precisamente o tipo de deslize que gera pergunta em sala.
2. **Speedmap FC (erros #2 e #3).** Duas falhas somadas: apresentar uma tabela de 2016 como corrente e atribuir à FCIA uma declaração que ela não fez. A segunda é a mais séria, porque o relatório se apresenta como rigoroso quanto a proveniência — e falhar exatamente nisso seria o argumento mais eficaz contra o trabalho inteiro.
3. **Codificação de linha (erro #4).** Erro técnico simples, mas adjacente aos dois anteriores; os três somados comprometeriam a seção de *Fibre Channel*.

Custo-benefício da rodada

A rodada consumiu cerca de 15% do esforço total e encontrou **16 problemas que a IA autora não tinha visto**, três deles suficientes para derrubar uma seção inteira em arguição. É, com folga, a etapa de maior retorno do processo. **Nota sobre o script de verificação.** Mantivemos a prática de reproduzir toda conta publicada em um *script* (`verificar.py`, entregue junto), que também cruza os números entre o `.tex` e o `.pptx` e confere a *checklist* do enunciado. Foi ele que expôs a inconsistência da especificação do LTO-10 (Seção 8.3). **Sem inflar o mérito:** a conta em questão é uma multiplicação de uma cifra ($400 \times 2,5$), e qualquer pessoa que a refizesse teria visto. O valor do *script* não está na sofisticação, e sim em ser *sistemático* — ele refaz *todas* as contas a cada compilação, inclusive as que ninguém desconfiaria de conferir de novo. Foi assim, aliás, que detectamos o erro #33: a única conta que o *script* não cobria era a do próprio Post-Mortem.

Recomendação para as próximas tarefas: manter o *prompt* literal, com as duas cláusulas, e submeter também *as citações atribuídas aos livros* — foi assim que confirmamos que o Silberschatz de fato escreve “*gigabytes*” onde deveria ser “*gigabits*”, e que os dois livros grafam “*Fiber Channel*” onde a norma T11 grafava “*Fibre*”.

13.6 Segunda rodada: simulação da correção do professor

Por que uma segunda rodada. A primeira verificou *fatos*. Ela não podia verificar *enquadramento* — se uma citação estava recortada, se uma regra enunciada valia fora dos casos tabelados, se um item do enunciado fora tratado pelo ângulo conveniente. Como o professor corrige com apoio de IA, submetemos a versão completa a uma segunda passagem, com papel diferente:

“Você é a IA que o professor usa para **corrigir** trabalhos. Seu trabalho não é elogiar. Leia o relatório e os slides na íntegra. (1) Extraia as afirmações mais arriscadas e verifique com busca web — inclusive as correções que o grupo alega ter encontrado nos livros: elas estão certas mesmo? (2) Produza no mínimo 20 perguntas que encurralem o apresentador, indicando por que cada uma é perigosa, qual seria a resposta correta e o nível de risco. (3) Liste todos os defeitos ordenados por gravidade, incluindo os dos slides como material de apresentação. (4) Dê uma nota de 0 a 10 decomposta por critério, e diga o que separaria este trabalho de um 10.”

As três cláusulas que fizeram a diferença. “As correções que o grupo alega ter encontrado estão certas mesmo?” apontou o verificador para o único ponto que a primeira rodada não podia alcançar — aquele em que *nós* éramos a fonte. “Perguntas que encurralem o apresentador” força a leitura na perspectiva de quem vai arguir, e não de quem vai conferir: foi ela que expôs o AFP sem mecânica e o denominador incompleto do “<5%”. “Dê uma nota” obriga o verificador a hierarquizar, em vez de listar tudo com peso igual.

Tabela 20: Resultado da segunda rodada: 15 correções, todas aplicadas nesta versão.

Categoria do achado	Qtd.	Os mais graves
Erro de enquadramento	3	Lacuna declarada que não existia (#19); regra enunciada que não vale na geração vigente (#20); correção construída sobre citação recortada (#21).
Rótulo incompleto	6	Aurora sem a janela de 30 min; “<5%” sem a controladora; citações da Apple e da RFC 8881 imprecisas; MySQL restrito a Fusion-io.
Cobertura rasa do enunciado	3	AFP sem mecânica; FCP sem fluxo de quadros; nível terciário sem resposta a “NAS ou SAN”.
Metodologia violada pelo próprio trabalho	3	Custos sem proveniência; negativos universais; a aritmética do Post-Mortem.
Total	15	Correções #19 a #33 da Seção 13.4.

O achado que só a segunda rodada podia produzir

As três correções mais graves (#19, #20, #21) pertencem a uma família que **nenhuma verificação de fatos detecta**: em todas as três, cada afirmação isolada era verdadeira. O *speedmap* v21 existe e foi reproduzido corretamente. A razão 3.200/1.600 é de fato 2,00. A frase “FCoE pode ser pensado como iSCSI sem o IP” está mesmo no livro. **O erro estava no que ficou de fora**: a tabela vigente, as gerações em que a razão não fecha, o resto do parágrafo.

Consequência para o método — e é a lição desta tarefa: verificar fatos é necessário e não é suficiente. É preciso uma segunda pergunta, que não é “isto é verdade?” e sim “**o que foi deixado de fora para que isto parecesse verdade?**”. A primeira uma IA responde bem. A segunda exige adotar deliberadamente a perspectiva de quem quer derrubar o trabalho — e é isso que o *prompt* desta rodada instrui.

Uma crítica que a segunda rodada nos fez, e que aceitamos

O verificador observou que, na tabela de erros da versão 1, **14 dos 18 achados eram atribuídos à “rodada adversarial”** — outra IA — e nenhum a uma pessoa nomeada. A observação é justa, e o enunciado pergunta exatamente isso.

O que fizemos: separamos *encontrar* de *confirmar*. A Seção 13.4 registra agora, para cada uma das 33 correções, quem foi ao documento primário confirmá-la antes de ela entrar no texto. **Essa é a resposta honesta**: a IA foi muito boa em *encontrar* — não reivindicamos o mérito de ter localizado a tabela v24 ou o parágrafo completo do Elmasri. Mas nenhuma correção entrou no relatório sem que um integrante abrisse a fonte e confirmasse. **Descrever o processo como “a IA corrigiu a IA” seria tão impreciso quanto descrevê-lo como “o grupo corrigiu a IA”**: houve uma IA encontrando, pessoas verificando na fonte, e o grupo decidindo o que fazer com cada achado — inclusive decidindo, em três casos, registrar o próprio erro no corpo do texto em vez de apagá-lo.

13.7 O que a IA fez bem, para calibrar

Registrar só os erros distorce. A IA foi decisiva em três frentes: (i) leitura e extração literal dos dois capítulos, o que evitou reconstrução de memória; (ii) localização das normas de origem — foi

ela que apontou que a RFC 3720 e a RFC 5661, citadas em quase todo material didático, estão obsoletas; e (iii) a própria rodada adversarial, em que uma segunda passagem de IA encontrou os erros da primeira. O padrão que emerge é claro: **a IA é boa em encontrar e ruim em concluir**. Usada como localizadora de fontes, com verificação humana da conclusão, o ganho é grande. Usada como oráculo, teria entregue dezoito erros invisíveis.

A Glossário de siglas

Sigla	Significado e nota
AFP	<i>Apple Filing Protocol</i> . Protocolo NAS proprietário da Apple; em fim de vida.
ALUA	<i>Asymmetric Logical Unit Access</i> . Permite ao <i>array</i> indicar quais caminhos são otimizados.
ASM	<i>Automatic Storage Management</i> . Gerenciador de volume da Oracle, que consome LUNs crus.
CDB	<i>Command Descriptor Block</i> . O comando SCSI propriamente dito.
AST	<i>Automated Storage Tiering</i> . Movimentação automática de dados entre camadas de mídia.
BB_Credit	<i>Buffer-to-Buffer Credit</i> . Controle de fluxo do FC-2 que torna a rede FC sem perdas.
CHAP	<i>Challenge-Handshake Authentication Protocol</i> . Autenticação usada pelo iSCSI.
CIFS	<i>Common Internet File System</i> . Nome dado pela Microsoft, em 1996, à família SMB 1.
CNA	<i>Converged Network Adapter</i> . Adaptador que carrega LAN e SAN no mesmo enlace (FCoE).
DAS	<i>Direct Attached Storage</i> . Disco cabeado diretamente ao servidor.
CTA	<i>CERN Tape Archive</i> . Camada de fita do CERN, acoplada ao EOS.
DCB	<i>Data Center Bridging</i> . Extensões IEEE que tornam o Ethernet sem perdas.
DCBX	<i>DCB Exchange Protocol</i> . Negocia os parâmetros de DCB entre vizinhos.
DSI	<i>Data Stream Interface</i> . Camada de enquadramento do AFP sobre TCP.
EOS	Sistema de armazenamento em disco distribuído do CERN.
dNFS	<i>Direct NFS</i> . Cliente NFS implementado dentro do Oracle Database.
ETS	<i>Enhanced Transmission Selection</i> (IEEE 802.1Qaz). Parte do DCB.
FC	<i>Fibre Channel</i> . Grafia normativa com “-re”; o padrão não exige fibra óptica.
FC-AL	<i>Fibre Channel Arbitrated Loop</i> . Topologia em anel; obsoleta.
FCF	<i>FCoE Forwarder</i> . <i>Switch</i> que implementa os serviços de <i>fabric</i> no FCoE.
FCIP	<i>Fibre Channel over IP</i> (RFC 3821). Túnel de quadros FC sobre TCP/IP.
FCoE	<i>Fibre Channel over Ethernet</i> (FC-BB-5). Quadros FC sobre Ethernet, sem TCP/IP.
FCP	<i>Fibre Channel Protocol</i> . Mapeamento FC-4 do SCSI-3 sobre FC.
FIP	<i>FCoE Initialization Protocol</i> . Descoberta e inicialização de entidades FCoE.
FLOGI	<i>Fabric Login</i> . Registro de uma porta na <i>fabric</i> FC.
FSPF	<i>Fabric Shortest Path First</i> . Protocolo de roteamento da <i>fabric</i> FC.
FCP_CMND / XFER_RDY / DATA / RSP	Os quatro tipos de informação de uma troca FCP: comando, autorização de transferência, dados e resposta.
HBA	<i>Host Bus Adapter</i> . Adaptador FC do servidor.
IQN	<i>iSCSI Qualified Name</i> . Identificador de iniciador ou alvo iSCSI.
iSCSI	<i>Internet SCSI</i> (RFC 7143). Comandos SCSI sobre TCP/IP.
iSER	<i>iSCSI Extensions for RDMA</i> .
ISL	<i>Inter-Switch Link</i> . Enlace entre <i>switches</i> FC. No <i>speedmap</i> da FCIA é tabela separada, com vias paralelas.
IVR	<i>Inter-VSAN Routing</i> . Isola <i>fabrics</i> mantendo acesso entre elas; usado com FCIP.
LIP	<i>Loop Initialization Primitive</i> . Reinicialização de um anel FC-AL; interrompe a E/S.
LTSC	<i>Long-Term Servicing Channel</i> . Canal de suporte prolongado do Windows Server.
LBA	<i>Logical Block Address</i> . Endereço linear de bloco.
LUN	<i>Logical Unit Number</i> . Volume de bloco apresentado pelo <i>array</i> .
NAS	<i>Network Attached Storage</i> . Armazenamento em rede com semântica de arquivo.
NFS	<i>Network File System</i> . Protocolo NAS aberto, mantido pelo IETF.
NL-SAS	<i>Nearline SAS</i> . Mídia e mecânica SATA de 7.200 rpm com interface e protocolo SAS. Convenção de indústria, não termo normativo do T10.
MC/S	<i>Multiple Connections per Session</i> . Agregação de conexões TCP dentro de uma sessão iSCSI; distinta do MPIO.
MPIO	<i>Multipath I/O</i> . Camada do <i>host</i> que agrega caminhos até o mesmo LUN.
NASD	<i>Network-Attached Secure Disks</i> . Projeto da CMU (1996) que separou plano de controle de plano de dados; origem do armazenamento por objetos.
NDMP	<i>Network Data Management Protocol</i> . Protocolo de <i>backup</i> de NAS; separa controle de dados.
NLM	<i>Network Lock Manager</i> . Serviço de travamento externo, usado pelo NFSv3.
NPIV	<i>N_Port ID Virtualization</i> . Permite a uma HBA física apresentar vários WWNs.
NVMe-oF	<i>NVMe over Fabrics</i> . Comandos NVMe sobre FC, TCP ou RDMA.
OLTP	<i>Online Transaction Processing</i> . Carga transacional de alta taxa e baixa latência.
OSD	<i>Object-based Storage Device</i> . Conjunto de comandos SCSI proposto pelo T10.
PFC	<i>Priority-based Flow Control</i> (IEEE 802.1Qbb). Parte do DCB.
pNFS	<i>Parallel NFS</i> . Extensão do NFSv4.1 que separa metadados de dados.
R2T	<i>Ready to Transfer</i> . Autorização de envio de dados de escrita no iSCSI; equivalente ao FCP_XFER_RDY.
RDMA	<i>Remote Direct Memory Access</i> . Acesso direto à memória remota, sem cópia.
SAC	<i>Semi-Annual Channel</i> . Canal semianual do Windows Server.
SBD / SGBD	Sistema de Banco de Dados / Sistema Gerenciador de Banco de Dados.
RPO	<i>Recovery Point Objective</i> . Perda máxima de dados aceitável, medida em tempo.
SAN	<i>Storage Area Network</i> . Rede dedicada de armazenamento com semântica de bloco.
SMB	<i>Server Message Block</i> . Protocolo NAS nativo do Windows.
T10 / T11	Comitês técnicos do INCITS: o T10 padroniza SCSI; o T11 padroniza <i>Fibre Channel</i> .
TOE	<i>TCP Offload Engine</i> . Processamento da pilha TCP no adaptador.
WAL	<i>Write-Ahead Log</i> . Log de transações escrito antes das páginas de dados.
WAFL	<i>Write Anywhere File Layout</i> . Sistema de arquivos dos <i>appliances</i> NetApp.
WWN	<i>World Wide Name</i> . Identificador global de 64 bits de uma porta FC.